

Automated machine learning for residential property valuation

Lin Deng , Xueqing Zhang 

Department of Civil and Environmental Engineering, The Hong Kong University of Science and Technology, Clear Water Bay, Kowloon, Hong Kong, China

ARTICLE INFO

Keywords:

Automated machine learning
Residential property valuation
Domain-specific
Domain-agnostic
Artificial intelligence implementation
Artificial intelligence application

ABSTRACT

Many studies have demonstrated that machine learning (ML) models surpass other approaches in residential property valuation. However, developing well-performing ML models requires the deep involvement of domain experts and data scientists, posing challenges for the scale of artificial intelligence (AI) implementation and application. Automated ML (AutoML) implements the end-to-end process of repetitive ML tasks without much human assistance. Although existing AutoML techniques have yielded promising results, no domain-specific AutoML framework is designed for residential property valuation. Most AutoML frameworks are domain-agnostic, overlooking domain knowledge, which limits their model performance and the practical applications of AI in real-world scenarios. This study proposes a domain-specific AutoML framework for residential property valuation (AutoML4RPV), which incorporates both domain-specific and domain-agnostic function modules to streamline AI implementation. Three real housing transaction datasets (New York, London, and Singapore) and a public valuation dataset are used for experimental validation by comparing AutoML4RPV with benchmark ML models and openly available AutoML frameworks. Integrating domain-specific modules, AutoML4RPV achieves the best performance with raw data input, outperforming the second-ranked domain-agnostic AutoML framework by 17.6 %, 18.2 %, and 9.80 % for the New York, London, and Singapore datasets, respectively. The domain-agnostic modules of AutoML4RPV are highly competitive and achieve the best model performance for the public valuation dataset. The data and codes of AutoML4RPV are openly available at <https://github.com/Linhkust/AutoML4RPV>.

1. Introduction

Property valuation is a process of estimating the true market value of a real estate property (Deppner et al., 2023). It is necessary for various stakeholders in the real estate market, including property sellers and buyers, real estate agencies, property developers, mortgage providers, and local authorities (Gao et al., 2022). Traditional property valuation models took a cost, income, or sale comparison approach (IAAO, 2018), considering property location, market conditions, and real estate characteristics. The hedonic pricing model is often used to depict the relationship between the property value and its influencing factors (Rosen, 1974) by multiple linear regression. Traditional models have limited capability, although they are well-explanatory and interpretable (Yacim and Boshoff, 2018). For example, the hedonic pricing model needs strict assumptions such as the normality of the residuals, homoscedasticity, independence, and non-multicollinearity (Alfaro-Navarro et al., 2020).

With the exponentially increasing volume of housing transactions, machine learning (ML) techniques have been applied to property

valuation (Deng, 2025; Deng and Zhang, 2025). ML models outperform traditional models in extracting information from big data, measuring the non-linear relationships between property value and its characteristics, and feature selection (Kang et al., 2021; Yoo et al., 2012). A detailed overview of machine learning for property valuation is provided in the review papers (Choy and Ho, 2023; Geerts et al., 2023; Tekouabou et al., 2023). A complete machine learning (ML) pipeline encompasses data preparation, feature engineering, model generation, and model application (He et al., 2021). It is time-consuming and laborious for *domain experts* without data science expertise (e.g., valuation practitioners) and *data scientists* without domain knowledge reserve (e.g., software engineers) to build the optimal ML pipeline for property valuation.

As is shown in Fig. 1, the AutoML approach can largely reduce ML model development costs by taking the place of humans in identifying (all or a part of) configurations of ML computer programs within a limited computational budget (Yao et al., 2018). However, current well-developed and open-source AutoML frameworks such as

* Corresponding author.

E-mail address: zhangxq@ust.hk (X. Zhang).

<https://doi.org/10.1016/j.engappai.2025.112035>

Received 1 January 2025; Received in revised form 2 July 2025; Accepted 12 August 2025

Available online 22 August 2025

0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

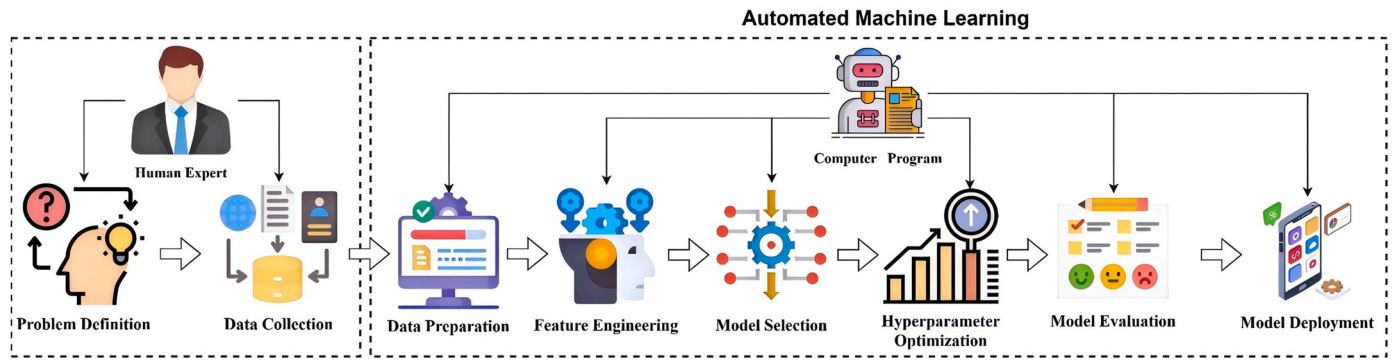


Fig. 1. Automated machine learning.

Table 1
Summary of open-source AutoML frameworks.

AutoML frameworks	Type	DC	DP	FE	MS	EL	HPO methods	XAI	MD	GUI
Auto-WEKA (Thornton et al., 2013)	DA	×	×	✓	✓	✓	Bayesian optimization	×	×	✓
Hyperopt-Sklearn (Komer et al., 2014)	DA	×	✓	✓	✓	×	Bayesian optimization	×	×	×
Auto-Sklearn (Feurer et al., 2015)	DA	×	✓	✓	✓	✓	Bayesian optimization	×	×	×
AutoCompete (Thakur and Krohn-Grimberghe, 2015)	DA	×	✓	✓	✓	×	Grid and random search	×	×	×
TPOT (Olson et al., 2016)	DA	×	×	✓	✓	✓	Genetic programming	×	✓	×
Auto-WEKA 2.0 (Kotthoff et al., 2017)	DA	×	×	✓	✓	✓	Bayesian optimization	×	×	✓
ATM (Swearingen et al., 2017)	DA	×	✓	✓	✓	×	Bayesian Tuning and Bandits	×	×	×
ML-Plan (Mohr et al., 2018)	DA	×	✓	✓	✓	✓	Hierarchical task networks	×	×	×
Auto-XGBoost (Thomas et al., 2018)	DA	×	×	✓	×	×	Bayesian optimization	×	✓	×
FLAML (Wang et al., 2019)	DA	×	×	×	✓	✓	Cost-frugal optimization and BlendSearch	✓	✓	×
H2O AutoML (Ledell, 2020)	DA	×	✓	✓	✓	✓	Grid search	✓	×	✓
AutoBazaar (Smith et al., 2020)	DA	×	✓	✓	✓	×	Bayesian Tuning and Bandits	×	×	×
AutoGluon (Erickson et al., 2020)	DA	×	✓	✓	✓	✓	Fixed default parameter (set adaptively)	✓	✓	×
PyCaret (Ali, 2020)	DA	×	✓	✓	✓	✓	Grid and random search	×	✓	×
mljar-supervised (Płońska, A., & Płoński, P., 2021)	DA	×	×	✓	✓	✓	Random Search	✓	×	×
AutoML for autonomous vehicles' risk prediction and behavior assessment (Shi et al., 2021)	DS	×	✓	✓	✓	✓	Bayesian optimization	×	×	×
Auto-Sklearn 2.0 (Feurer et al., 2022)	DA	×	✓	✓	✓	✓	Successive halving and Bayesian optimization	×	×	×
AMLBID (Garouani et al., 2022)	DA	×	✓	✓	✓	×	Grid and random search	✓	✓	✓
BioAutoML (Bonidia et al., 2022)	DS	×	×	✓	✓	✓	Bayesian optimization with TPEs	×	×	×
AutoML for minerals processing data (Koh et al., 2022)	DS	×	✓	✓	✓	×	Evolutionary algorithm	×	✓	×
AutoML for earthquake casualty rate and economic loss prediction (W. Chen and Zhang, 2022)	DS	✓	✓	✓	✓	×	Bayesian optimization	✓	×	×
AMLBID 2.0 (Garouani et al., 2023)	DA	×	✓	✓	✓	×	Grid and random search	✓	✓	✓
Greedy-AutoML (Sahin and Demir, 2023)	DS	×	✓	✓	✓	✓	Grid search	×	×	✓
HAMLET (Francia et al., 2023)	DA	×	✓	✓	✓	×	Bayesian optimization with cost-frugal optimization	×	✓	✓
AutoSL (Mohan and Badra, 2023)	DA	×	×	×	✓	✓	Bayesian optimization and genetic algorithm	×	✓	×
Geochemistry π (ZhangZhou et al., 2024)	DS	×	✓	✓	✓	×	Cost-frugal optimization and blended search strategy	×	✓	✓

mjar-supervised (Płońska and Płoński, 2021) and H2O (Ledell, 2020) cannot be directly used for property valuation because they require users to prepare clean and machine-readable datasets in advance and lack key procedures, including data collection and preparation. Although some AutoML frameworks support raw data input and automatic feature extraction and generation, such as PyCaret (Ali, 2020) and AutoGluon (Erickson et al., 2020), they fail to incorporate domain knowledge into the ML pipeline and generate meaningless and incomprehensible features. AutoML has been widely used in various domains such as healthcare (Waring et al., 2020), industrial big data (Garouani et al., 2022), time series forecasting (Alsharif et al., 2022; Meisenbacher et al., 2022), multi-label classification (Wever et al., 2021), spatial decision support system (Wen and Li, 2023), and neural architecture search (Baymurzina et al., 2022; Gudzius et al., 2023; Jaafra et al., 2019), and there is no domain-specific AutoML framework for residential property valuation yet.

The absence of a specialized AutoML framework for residential

property valuation represents a critical research gap, as conventional domain-agnostic AutoML solutions fail to address the fundamental challenge, i.e., the knowledge fusion between domain experts knowledgeable in real estate valuation principles and data scientists skilled in technical capabilities during AutoML implementation. This paper proposes a specialized AutoML framework for residential property valuation, i.e., AutoML4RPV, to address this gap by incorporating domain-specific (i.e., data collection and preparation and feature engineering) and domain-agnostic (i.e., model generation and evaluation, model interpretation, and deployment) functions to enhance both the efficiency of AI-based property valuation model development and the accuracy of property valuation predictions.

The major contributions of this study are: (1) proposing a specialized AutoML framework that leverages the expertise of both domain experts and data scientists to build ML models for property valuation from scratch; (2) incorporating domain knowledge about property valuation into the AutoML framework by building domain-specific function

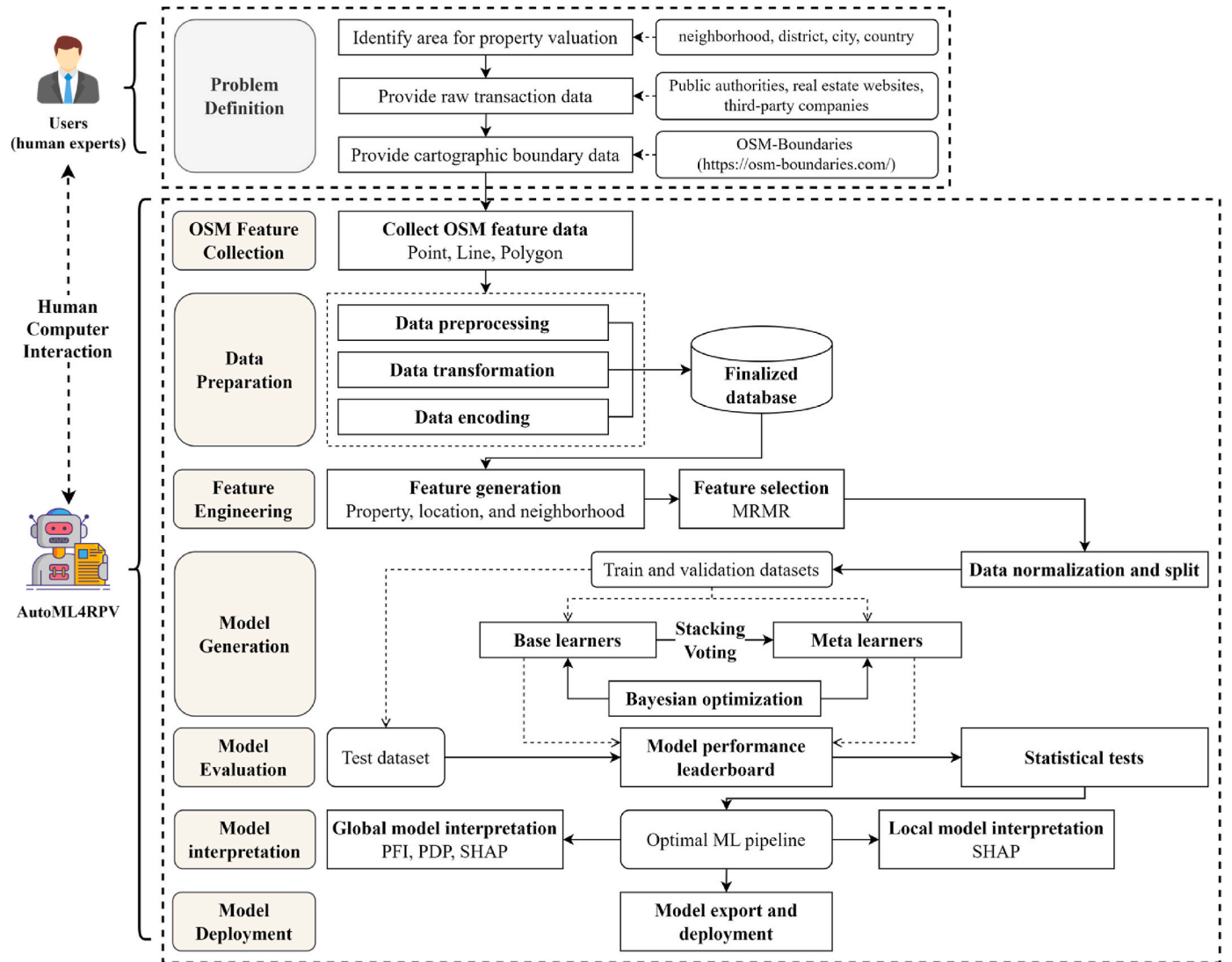


Fig. 2. Flowchart of the proposed AutoML4RPV framework.

modules; (3) designing a novel domain-agnostic model training strategy based on Minimum Redundancy Maximum Relevance (MRMR) feature selection method and ensemble learning techniques; (4) developing a Python *Shiny* platform based web app that provides users with easy access to AutoML4RPV.

The rest of the paper is organized as follows. Section 2 reviews the literature on property valuation and available AutoML frameworks. Section 3 introduces the proposed AutoML framework. Section 4 presents the experimental validation procedures for the proposed AutoML4RPV and benchmarks it against other AutoML techniques using three real housing transaction datasets and one public valuation dataset. Section 5 discusses the experimental validation results and implications of this study. Section 6 presents the limitations and future research directions. Section 7 concludes the paper.

2. Literature review

2.1. ML-based property valuation

The hedonic pricing model (HPM) depicts the relationship between property value and its internal and external influencing factors (Rosen, 1974). Internal factors are related to the property characteristics, such as view orientation, number of rooms, floor level and area, building age,

etc. External factors include macro-level and micro-level. Macro-level factors are primarily time-dependent housing market indicators, including inflation, interest rates, gross domestic product (GDP), unemployment, housing supply and demand (Soltani et al., 2021). Micro-level factors are primarily related to the property's geographic location (Soltani et al., 2022), including geographic coordinates, distance to the city center or central business districts (CBD), neighborhood amenities, sociodemographic status, crime rate, built and natural environment, among others.

Among various ML models used for property valuation, tree-based models typically perform the best due to ensemble learning, which utilizes a weighted combination of multiple base learners to solve classification or regression problems (Sagi and Rokach, 2018). Ensemble learning includes four representative strategies: (1) **Boosting** that integrates a set of weak learners into a strong learner to minimize training errors (Schapire, 1990); (2) **Bagging** that uses the bootstrap sampling method to draw training data and calculates an aggregated value of base learner's predictions as the final prediction (Breiman, 1996); (3) **Stacking** that learns from learners in multiple learning stages (Wolpert, 1992); (4) **Voting** that produces the weighted sum of different base models' predictions. Despite the excellent performance of ensemble learning, most studies apply ensemble learning to property valuation using individual models, such as bagging-based random forests (RF) and

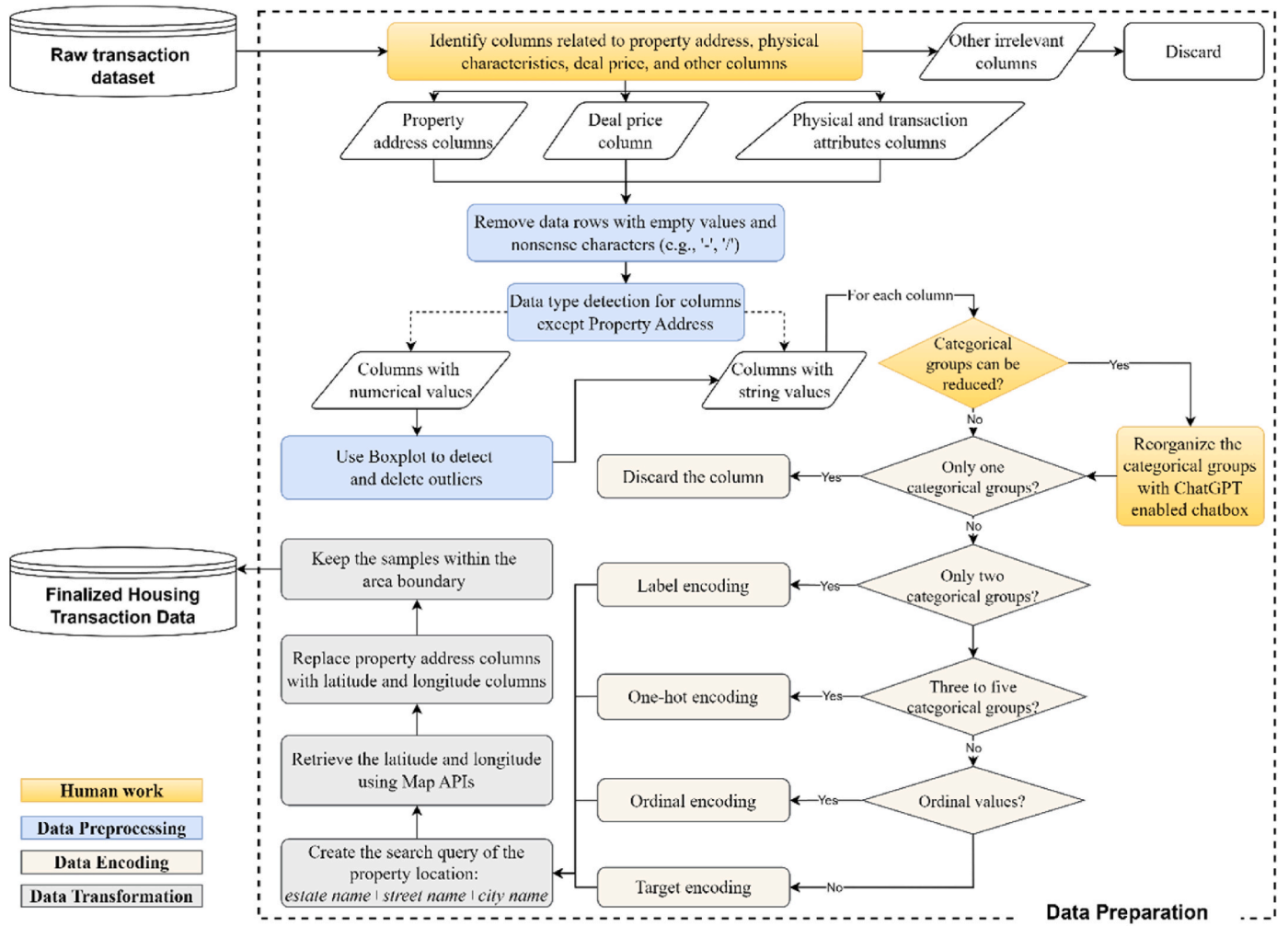


Fig. 3. The flowchart of data preparation.

boosting-based XGBoost. Only a few studies adopted stacking and voting in housing price prediction (Lim and Bellotti, 2021; Xu and Li, 2021; Zhan et al., 2023). The results show that stacking and voting methods outperform individual ML models. Therefore, when building ML pipelines for property valuation, it is advisable to consider a comprehensive ensemble learning strategy that incorporates bagging, boosting, stacking, and voting.

2.2. AutoML frameworks

Table 1 summarizes the available open-source AutoML frameworks regarding whether they are domain-agnostic (DA) and domain-specific (DP), whether they support data collection (DC), data preparation (DP), feature engineering (FE), and model selection (MS), whether they support ensemble learning (EL) methods, used hyperparameter optimization (HPO) methods, whether they support explainable artificial intelligence (XAI), model deployment (MD) and graphic user interface (GUI).

AutoML frameworks in earlier studies were predominantly domain-agnostic, irrespective of the domains to which the datasets belong. Users only need to feed their processed data to the AutoML frameworks for automatic model training. Almost half of the frameworks do not support comprehensive ensemble learning methods and data preparation. Few studies incorporate data collection into their frameworks, except for Chen and Zhang (2022). All AutoML frameworks consider FE and MS, except for AutoSL that excludes FE procedures (Mohan and

Badra, 2023). The AutoML frameworks widely use the Bayesian Optimization method for hyperparameter optimization. Only two studies provide model interpretation, deployment functions, and GUIs (Garouani et al., 2022, 2023; Garouani et al., 2022a,b). In this study, our proposed AutoML4PV will fill the gaps by incorporating both domain-specific and domain-agnostic functions, supporting comprehensive ensemble learning methods, XAI, model deployment, and a GUI.

3. Proposed AutoML4PV framework

Fig. 2 shows the flowchart of the proposed AutoML4RPV framework. Users (human experts) define the problem first by identifying the area for property valuation (e.g., neighborhood, district, city, and country), providing raw housing transaction datasets collected from public authorities, real estate websites, and third-party companies, and providing cartographic boundary data of the area from OSM-Boundaries (<https://osm-boundaries.com/>). Then, users can send the data to our AutoML4PV system for automatic OSM feature collection, data preparation, feature engineering, model generation and evaluation, model interpretation, and model deployment. A detailed introduction of each function module is provided in the following subsections. An online web app has been developed based on the AutoML4RPV framework, enabling users to build their own residential property valuation model from scratch. The tutorial for the web app is provided in the Appendix.

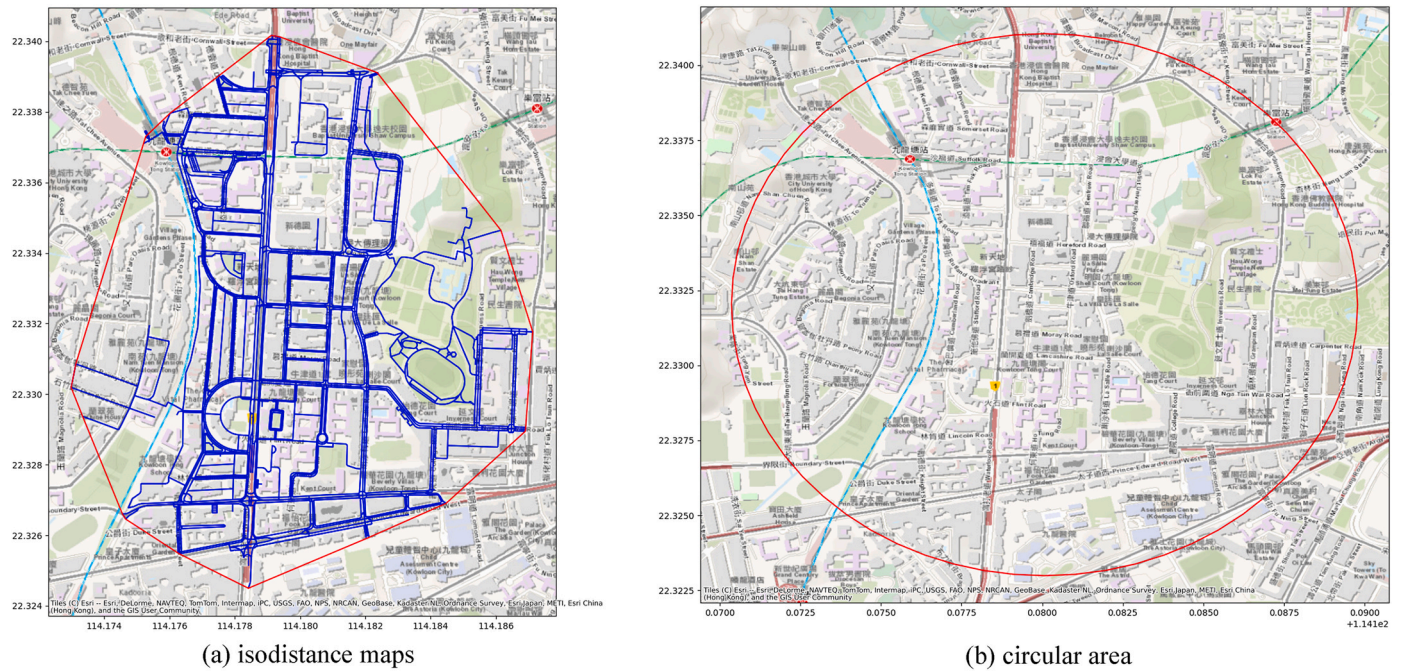


Fig. 4. Illustration of (a) isodistance maps and (b) circular area.

3.1. Domain-specific data manipulation and feature generation

3.1.1. OSM data collection

Once users provide cartographic boundary data for property valuation, AutoML4PV can automatically retrieve the OpenStreetMap (OSM) feature data in this area. The OSM database offers an extensive amount of free spatial data, which is continually updated and growing (Ramm, 2014). OSM feature layers include point layers (*places, points of interest, places of worship, natural, traffic, and transport*), line layers (*roads, railways, and waterways*), and polygon layers (*buildings, land use, and water bodies*). The OSM features may include irrelevant regions, and a cartographic boundary is needed to narrow the OSM data coverage to specific areas, such as neighborhoods, districts, cities, and countries.

3.1.2. Data preparation

Fig. 3 presents the flowchart of data preparation. Based on the raw transaction datasets, users identify the columns related to the property address, physical and transaction attributes, and deal price. Other irrelevant columns are discarded. Data preparation includes data preprocessing, encoding, and transformation.

- (1) **Data preprocessing.** Data rows with empty values and nonsense characters are removed. The data type of each column (except Property Address columns) will be detected and categorized into columns with numerical and string values, respectively. The boxplot approach is used to detect and delete the outliers in the columns with numerical values.
- (2) **Data encoding.** Columns with string values are first checked to determine whether the categorical groups in these columns can be reduced. If so, our proposed AutoML4PV enables users to utilize ChatGPT to formulate Python codes to reorganize the categorical groups in these columns. Columns with only one group are discarded. Label encoding is used to deal with columns with only two categories: 1 for one option and 0 for the other. One-hot encoding is applied when the columns contain three to five categories. Ordinal encoding is used if there are more than five categories in the column and the categories are identified as ordinal. Target encoding is applied to the columns by replacing a

categorical value with the weighted value of the target mean for the category and global target mean (Micci-Barreca, 2001).

- (3) **Data transformation.** After preprocessing and encoding data, our proposed AutoML4PV will transform the property address into geographic coordinates, i.e., latitude and longitude. The search query string of the property address is created in the format of “*estate name + street name + city name*”. Map APIs, such as Nominatim’s search API and Google Maps’ geocoding API, are used to retrieve the geographic coordinates of the property address. *Nominatim* is a free and open-source geocoder, and *Google Maps* requires users to provide API keys for quota-based geocoding services. The property address columns are then replaced with latitude and longitude columns. The area boundary is used to keep the geocoded housing samples within the regions in case some of the geographic coordinates are inaccurate.

In the data preparation stage, the most time-consuming procedure is geocoding, which involves transforming a property address into its corresponding latitude and longitude using the Map Geocoding API. In this study, the *Nominatim* API is used by default for geocoding because it is a free API with no charge for requests. However, it has a strict usage policy for service requests, including an absolute maximum of one request per second, limiting requests to a single thread, and restricting them to a single machine. These limitations make the geocoding process for large datasets rather time-consuming. Therefore, we extracted the unique addresses from the housing transaction records, as one unique property address could correspond to multiple housing transactions, which would largely reduce the number of geocoding requests. AutoML4RPV also supports the Google Maps Geocoding API, which can be used for large-scale datasets as it has no limit to daily requests and a higher usage limit (3000 queries per minute). However, it will charge fees if users exceed their quotas.

3.1.3. OSM-based feature generation

Based on the availability of data and the OSM database, AutoML4RPV defines three types of features: property, location, and neighborhood. Property features refer to the characteristics of the housing structure and transaction attributes. Location features are related to the property’s geographic location. Neighborhood variables

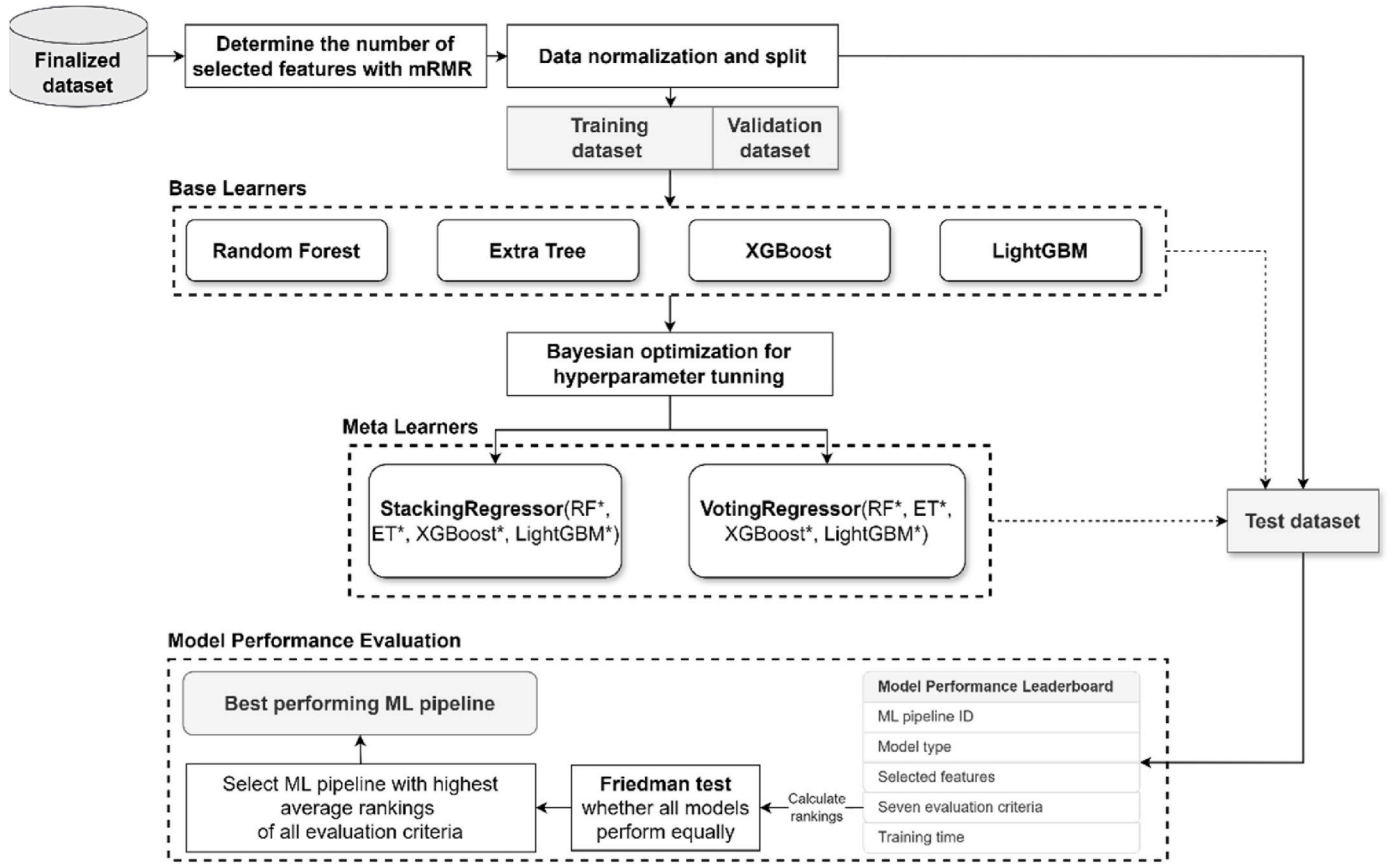


Fig. 5. Procedures of domain agnostic ML pipeline generation and evaluation.

describe the surrounding characteristics of the property. Based on previous studies on significant housing price determinants, AutoML4RPV generates neighborhood features based on the point, line, and polygon features in the OSM database:

- (1) **Point feature density, diversity, and accessibility** are measured for point features. The point feature density P_{ix} is defined as the proportion of OSM point feature i within a pre-defined area of property x (denoted as A_x). A_x can be an iso-distance map that indicates areas that fall within a certain network distance from a given point (Fig. 4a) or a circular area that uses a certain Euclidean distance from a given point as the radius (Fig. 4b). Point feature diversity D_{xi} refers to the number of point feature types within A_x . Point feature accessibility is measured as the nearest network distance or Euclidean distance between the property and the closest point feature, such as bus stops, subway entrances, etc.
- (2) **Line accessibility** is measured for line features as the Euclidean distance between the property and the nearest line feature, such as the main road, railway tracks, etc.
- (3) **Polygon density and accessibility** are measured for polygon features. The polygon density B_{ix} is defined as the proportion of building area in A_x . The polygon accessibility is measured as the nearest network distance or Euclidean distance between the property and the closest polygon feature, such as urban green and blue space.

In the feature engineering stage, computational costs are largely dependent on the feature generation configurations and the data size. For instance, calculating the Euclidean distance between the property and the nearest facility is less time-consuming than network distance. Large data samples will increase the computational costs of feature

engineering. In this study, based on the extracted unique property address, the Python *pyogrio* package was first used to provide fast, bulk-oriented read and write access to vector data sources, such as OSM data in the shapefile format and cartographic boundary data in the GeoJSON format. AutoML4RPV also utilized the Python *multiprocessing* package to enable parallel computing, thereby speeding up the OSM feature generation process. Each core is responsible for generating the OSM features of one data sample.

3.2. Domain agnostic ML pipeline generation and evaluation

The domain-agnostic ML pipeline generation and evaluation procedures are shown in Fig. 5. Based on the finalized dataset, the number of selected features is first determined using the MRMR method. The z-score method is used for data normalization, and the normalized data is split into training, validation, and test datasets. Four ensemble learning models are selected as base learners: Random Forest, Extra Tree, XGBoost, and LightGBM. Bayesian optimization is used to optimize the hyperparameters of the base learners. The optimized base learners are further used to build two meta-learners: one stacking model with default hyperparameters and one weighted voting model using the cross-validation loss of four optimized base learners as weights. Each trained model is applied to the test dataset to formulate a model performance leaderboard, which contains information such as ML pipeline ID, model types, selected features, model performance, and training time. The Friedman test is used to calculate the rankings of each ML pipeline based on seven evaluation criteria and determine whether all pipelines perform equally. The ML pipeline with the highest average rankings is selected as the best-performing one.

3.2.1. MRMR-based feature selection

Feature selection removes redundant variables to improve prediction

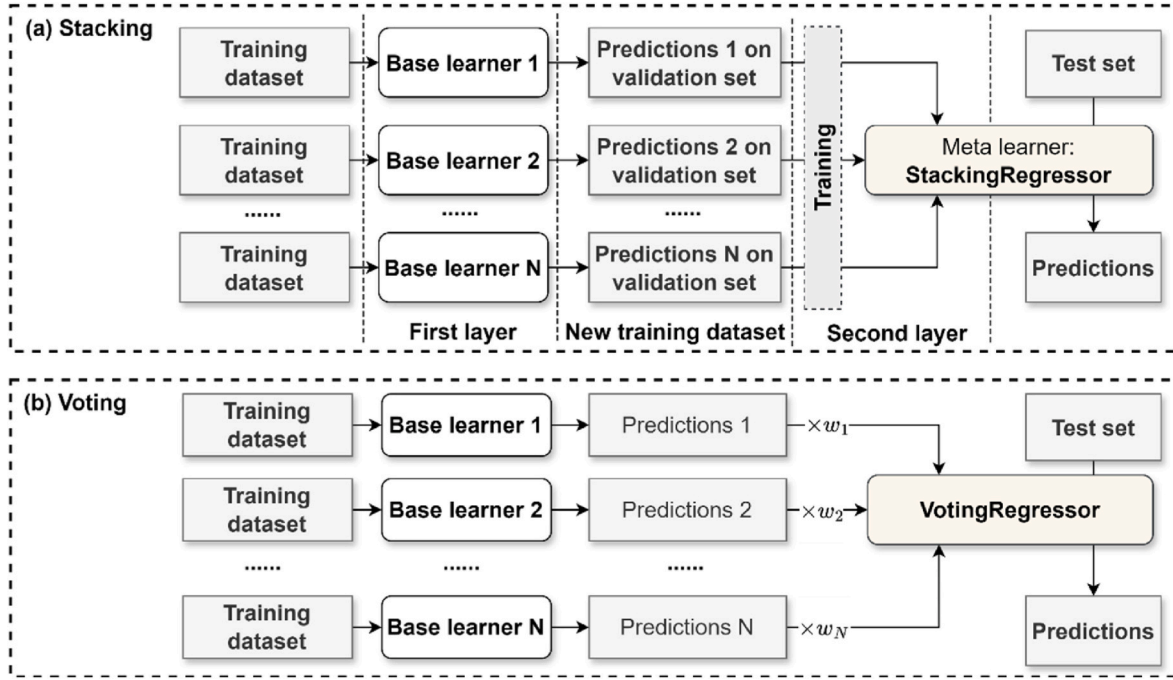


Fig. 6. Two types of ensemble strategies: (a) stacking and (b) voting.

Table 2
Hyperparameter search space.

Model	Selected hyperparameters	Search space ^a
RF	Number of trees in the forest ($n_estimators$)	Range (50, 150, 1)
	Maximum depth of the tree (max_depth)	Range (10, 100, 1)
ET	Number of trees in the forest ($n_estimators$)	Range (50, 150, 1)
	The maximum depth of the tree (max_depth)	Range (10, 100, 1)
LightGBM	Maximum number of estimators ($n_estimators$)	Range (50, 150, 1)
	Maximum tree depth for base learners (max_depth)	Range (10, 100, 1)
	Maximum tree leaves for base learners (num_leaves)	Range (10, 50, 1)
XGBoost	Number of gradient-boosted trees ($n_estimators$)	Range (50, 150, 1)
	Maximum tree depth for base learners (max_depth)	Range (10, 100, 1)

Note.

^a The values of search space from left to right refer to the lower bound of the hyperparameter, the upper bound of the hyperparameter, and the step size of the search.

performance (Chandrashekar and Sahin, 2014). The Minimum Redundancy Maximum Relevance (MRMR) method selects the most efficient and distinctive features. MRMR uses mutual information to find the optimal feature set by minimizing the redundancy between selected features and maximizing the relevance between selected features and target variable simultaneously (Peng et al., 2005) with Eq. (1):

$$\max \Phi(S) = D - R = \frac{1}{|S|} \sum_{x_i \in S} I(x_i, y) - \frac{1}{|S|^2} \sum_{x_i, x_j \in S} I(x_i, x_j) \quad (1)$$

where D is the relevance between the selected feature set S and the target variable y . R is the redundancy between the selected features in S . $I(\cdot)$ is the mutual information operator. AutoML4RPV provides three options for MRMR feature selection configuration: 1) *Best quality*. The number of

feature configurations to select is the total number of features. 2) *High quality*. The number of feature configurations to select is 60 % of the total number of features. 3) *Good quality*. The number of feature configurations to select is 30 % of the total number of features. The procedures of the MRMR algorithm are shown in Algorithm 1.

Algorithm 1. Minimum Redundancy Maximum Relevance

Input: Dataset D , feature set $\mathcal{F}$, number of selected features q

Output: A feature subset $\{f_1, f_2, \dots, f_q\} \subseteq \mathcal{F}$

```

1:  $S = \emptyset$ 
2: for  $k = 1, 2, \dots, q$  do
3:    $f_k \in \operatorname{argmax}\{\Phi(f) | f \in \mathcal{F}/S\}$ 
4:   Add  $f_k$  to  $S$ 
5: end for
6: return  $S$ 

```

3.2.2. Model selection

Two bagging models (Random Forest and Extra Tree), two boosting models (XGBoost and LightGBM), and two ensemble learning strategies (two-layer stacking and weighted voting) are incorporated in our AutoML4PV.

Four base learners: Random Forest uses a new dataset with a replacement (bootstrap sampling) from an existing dataset and trains several decision tree models with randomly selected features on the new dataset (Breiman, 1996). The predictions of each decision tree model are aggregated into the final prediction. Extra Trees splits nodes by choosing cut points randomly and using the whole learning sample (rather than a bootstrap replica) to grow the trees (Geurts et al., 2006). XGBoost is a gradient boosting tree (GBDT) algorithm that constructs the objective function of model deviation and regularization terms to prevent overfitting (T. Chen and Guestrin, 2016). LightGBM is also a GBDT-based algorithm that proposes two novel techniques, gradient-based one-side sampling (GOSS) and exclusive feature bundling (EFB), to realize faster training efficiency and lower memory usage (Ke et al., 2017). **Two meta learners:** Stacking uses the predictions of base learners as a new training dataset and uses a meta learner to learn the data (Fig. 6a). Voting fits the base learners on the original dataset and obtains a weighted average of

each base learner's predictions (Fig. 6b). The model training in AutoML4RPV is based on the Python *scikit-learn* package, and it has *fit* and *predict* functions and supports parallel computing by setting the parameters of *n_jobs*.

3.2.3. Bayesian optimization

Bayesian optimization (BO) is an iterative stochastic optimization framework for the combined algorithm selection and hyperparameter (CASH) optimization problem (Bischl et al., 2023). It first builds a probabilistic surrogate model (Gaussian process or tree-based model) mapping from the hyperparameters to the objective metrics. Then, it defines an acquisition function to decide which hyperparameter configuration to evaluate next, balancing the exploration and exploitation during the search process (Frazier, 2018). The hyperparameter search space of the four base learners is presented in Table 2. The search steps for all hyperparameters are set as one.

BO is computationally expensive, so we employed the parallelization technique and early stopping mechanisms to accelerate the HPO process using the Python *scikit-optimization* package. Early stopping helps prevent overfitting and reduces the computational cost of training by stopping the training process when the model performance on a validation set stops improving. The number of iterations is set as a threshold: if the model fails to improve on the validation set for the number of iterations, the training process is stopped.

3.2.4. Evaluation criteria

The trained ML models are compared against multiple evaluation criteria to identify the best performing one: root mean squared error (RMSE) (Eq. (2)), normalized RMSE (NRMSE) (Eq. (3)), mean absolute error (MAE) (Eq. (4)), mean absolute percentage error (MAPE) (Eq. (5)), R squared (Eq. (6)), and coefficient of dispersion (COD) (Eq. (7)).

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2)$$

$$NRMSE = \frac{\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}}{\bar{y}} \times 100 \quad (3)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4)$$

$$MAPE = \frac{1}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|} \times 100 \quad (5)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i \quad (6)$$

$$COD = \frac{100\%}{n} \sum_{i=1}^n |r_i - \tilde{r}| \quad (7)$$

where y_i = the actual value, $\hat{y}_i$ = the predicted value, $\bar{y}$ = the mean of the actual values, COD = coefficient of dispersion, $r_i = \hat{y}_i / y_i$, and $\tilde{r}$ = the median of r_i in a dataset with n numbers of samples.

3.2.5. Statistical test

The Friedman test is a non-parametric test used to determine whether there are any significant differences in model performance between ML pipelines on the leaderboard. It ranks the performance of each pipeline to obtain an average ranking and calculate the Friedman statistics (Friedman, 2001):

$$T_{\chi^2} = \frac{12d}{d(d+1)} \sum_{j=1}^k \left(R_j - \frac{k+1}{2} \right)^2 \quad (8)$$

where d is the number of evaluation criteria, and k is the number of ML pipelines. R_j is the average ranking of pipeline j . The null hypothesis H_0 of Friedman test indicates all model performances are non-significant. T_{χ^2} is roughly distributed as a χ^2 distribution with $k-1$ degrees of freedom under the assumption that the original hypothesis H_0 retains. The ML model with the highest average ranking is selected as the best-performing one and used for further analysis of model interpretability. The Nemenyi post-hoc test is applied to further distinguish between the algorithms (Nemenyi, 1963). In this process, the performance of the two models will be significantly different when their average rank difference exceeds a certain critical distance:

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6d}} \quad (9)$$

where the critical value q_α is the studentized range statistic. k is the number of models, and d is the number of evaluation metrics or datasets.

3.3. Model interpretation analysis

XAI methods such as permutation feature importance (PFI), partial dependence plots (PDP), and Shapley Additive exPlanations (SHAP) are applied for model explainability analysis.

PFI is a global model-agnostic model interpretation method designed to find important features that determine predictive performances. PFI is represented as the increase in the model's prediction error after permuting the feature's values. The more prediction errors the permutation causes, the more significant the feature. The PFI for feature j is computed as follows:

$$PFI_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j} \quad (10)$$

where s is the reference score of the model on data (e.g., the R squared for a regressor). K is the total repetition times of random shuffling. $s_{k,j}$ is the score on the dataset after k th repetition on shuffling the data of feature j .

PDP can visually describe how a specific input feature influences the ML model predictions by revealing the individual (one-dimensional PDP) or combined impacts (two-dimensional PDP) of specific features on the predictions while holding other features constant (Friedman, 2001). The partial dependence function for given value(s) of features S shows the average marginal effect on the prediction and is estimated by calculating averages in the training data (Molnar, 2020):

$$\hat{f}_S(x_S) = E_{X_C}[\hat{f}(x_S, X_C)] = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_S, x_C^{(i)}) \quad (11)$$

The x_S are the features for which the partial dependence function should be plotted and X_C are the other features used in the ML model $\hat{f}$. The partial function $\hat{f}_S$ is estimated with the Monte Carlo method by calculating averages in the training data. $x_C^{(i)}$ are actual feature values of other features in which we are not interested, and n is the number of instances in the dataset.

SHAP is a model-agnostic technique used for both local and global model interpretation. For an individual sample, SHAP can explain its prediction by computing the SHAP values of each feature. SHAP values are Shapley values based on coalitional game theory (Lundberg and Lee, 2017). The SHAP value for feature j of observation x , $\phi_j(x)$, is defined as:

$$\phi_j(x) = \sum_{S \subseteq \{1,2,\dots,M\} \setminus \{j\}} \frac{|S|!(M-|S|-1)!}{M!} [f(x_S \cup \{j\}) - f(x_S)] \quad (12)$$

Table 3
Overview of the datasets.

Feature Name	Feature Descriptions	Data Type
New York Dataset		
BOROUGH	The name of the borough in which the property is located.	Integer
NEIGHBORHOOD	The name of the neighborhood in which the property is located.	String
BUILDING CLASS CATEGORY	Building class (e.g., One Family Dwellings)	String
TAX CLASS AT PRESENT BLOCK	Tax class	String
LOT	Sub-division of the borough on which real properties are located	Integer
BUILDING CLASS AT PRESENT	Subdivision of a Tax Block and represents the property's unique location	Integer
ADDRESS	Detailed building class (e.g., A5 in One Family Dwellings)	String
Zip Code ZIP CODE	The property's street address	String
RESIDENTIAL UNITS	The property's postal code	Integer
COMMERCIAL UNITS	The number of residential units at the listed property	Integer
TOTAL UNITS	The number of commercial units at the listed property	Integer
LAND SQUARE FEET	The total number of units at the listed property	Integer
GROSS SQUARE FEET	The land area of the listed property in square feet	Integer
YEAR BUILT	The total area of all the floors of a building as measured from the exterior surfaces of the outside walls of the building, including the land area and space within any building or structure on the property	Integer
TAX CLASS AT TIME OF SALE	Year the structure on the property was built	Integer
BUILDING CLASS AT TIME OF SALE	Tax class when the property is sold	Integer
SALE PRICE	Building class when the property is sold	String
SALE DATE	Price paid for the property	String
	Date the property sold	Date
London Dataset		
Transaction unique identifier	A reference number which is generated automatically recording each published sale.	String
Price	Sale price	Integer
Date of Transfer	Date when the sale was completed, as stated on the transfer deed.	Date
Postcode	Postcode used at the time of the original transaction.	String
Property Type	Types of properties (D = Detached, S = Semi-Detached, T = Terraced, F = Flats/ Maisonettes, O = Other)	Character
Old/New	Age of the property (Y = a newly built property, N = an established residential building)	Character
Duration	Tenure of the property (F = Freehold, L = Leasehold)	Character
PAON	Primary addressable object name: house number or name	String
SAON	Secondary addressable object name: unit or flat number	String
Street	Street name	String
Locality	Locality name	String
Town/City	Town/city name	String
District	District name	String
Country	Country name	String
PPD Category Type	Type of Price Paid transaction	Character
Record Status - monthly file only	Additions, changes and deletions to the records (A = Addition, C = Change, D = Delete)	Character
Singapore Dataset		
Project Name	Property name	String
Transacted Price (\$)	The purchase price of the property	Integer

Table 3 (continued)

Feature Name	Feature Descriptions	Data Type
Area (SQFT)	The floor area of the property in square feet	Float
Unit Price (\$ PSF)	Transacted Price/Area (in SQFT)	Integer
Sale Date	Date when the property is sold	Date
Street Name	Street name	String
Type of Sale	Sale type (New sale, resale, sub-sale)	String
Type of Area	Area type that indicates whether the area pertains to the transacted property (land or strata)	String
Area (SQM)	The floor area of the property in square meters	Float
Unit Price (\$ PSM)	Transacted Price/Area (in SQM)	Float
Nett Price(\$)	The purchase price of a unit that is net of any upfront direct discounts and the value of any benefits given or agreed to be given for units sold.	Integer
Property Type	Property type (apartment or condominium)	String
Number of Units	Number of housing units in the project	Integer
Tenure	Tenure of the property	String
Postal District	Postal district ID	Integer
Market Segment	Location of the property (Core Central Region, Rest of Central Region and Outside Central Region)	String
Floor Level	The floor level of the property	String

where S is a subset of the features with feature j excluded. M is the total number of features. f is the model's prediction function. Furthermore, the global feature importance of a feature is calculated by averaging absolute Shapley values. Significant features have larger absolute Shapley values. SHAP feature importance (FI) for feature j is calculated as (Molnar, 2020):

$$SHAP_j = \frac{1}{n} \sum_{i=1}^n |\phi_j(i)| \quad (13)$$

where $|\phi_j(i)|$ is the absolute value of SHAP value for feature j of observation i .

4. Experimental validation

Given the data availability and Global Financial Centers Index (GFCI) 34 ranking results (Wardle and Mainelli, 2023), three leading international financial centers are selected for the experimental validation of our proposed AutoML4RPV framework: New York, London, and Singapore. Hong Kong ranks third but has no public housing transaction data source, so it is not selected. Housing transaction records of these study areas in 2022 were collected, including (1) 93,427 records in New York, (2) 63,092 records in London, and (3) 19,652 records in Singapore. New York and London provide web links for data retrieving. Singapore offers a web platform that allows users to set search filters, browse, and export data. Table 3 presents an overview of the three datasets. A desktop with an 8-core AMD Ryzen 7 5700X processor (3.40 GHz) and 32 GB of RAM was used for conducting the experiments.

4.1. Experiment validation with AutoML4RPV

New York: Housing transaction data in New York are stored in five separate Excel documents (Manhattan, Bronx, Brooklyn, Queens, Staten Island). All the five documents are copied into a new Excel document. Physical and transaction attributes include *RESIDENTIAL UNITS*, *COMMERCIAL UNITS*, *GROSS SQUARE FEET*, *YEAR BUILT*, *TAX CLASS AT TIME OF SALE*, *BUILDING CLASS AT TIME OF SALE*. Property address columns include *BOROUGH*, *BLOCK*, *LOT*, *ADDRESS*, *ZIP CODE*. The price column is *SALE PRICE*. There are two columns with texts: (1) *BUILDING CLASS AT TIME OF SALE*. Ordinal encoding and column

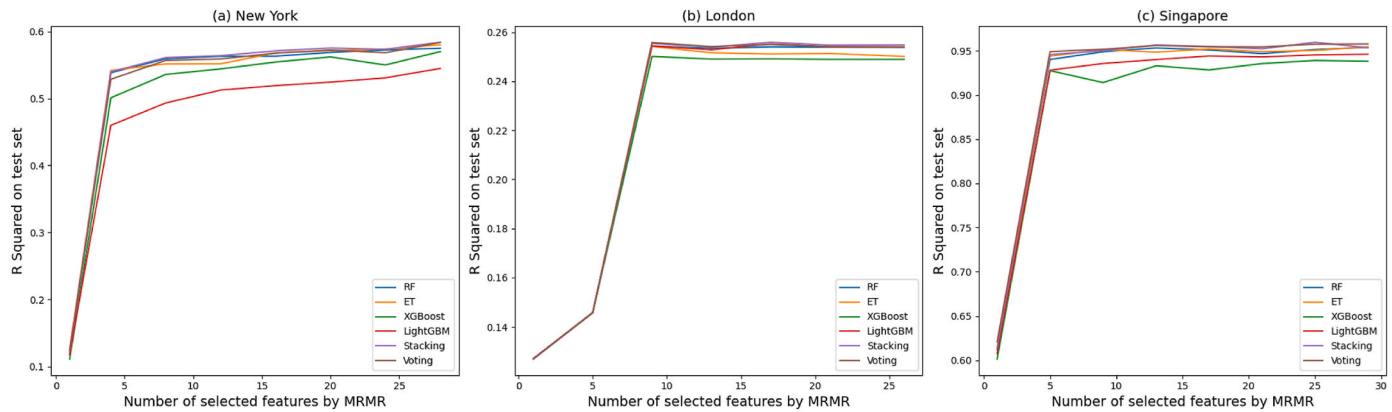


Fig. 7. Model performance with the different number of features selected by MRMR: (a) New York, (b) London, and (c) Singapore.

Table 4
Performance comparison of different models with the highest average ranking in AutoML4RPV.

Evaluation metrics	RMSE	NRMSE	MAE	MAPE	R2	COD
New York Dataset						
SVR	168694.3465	22.45 %	132601.8406	19.26 %	0.0458	16.9956
ANN	127423.9313	16.96 %	96893.6143	12.80 %	0.4556	13.7272
KNN	113840.8014	15.15 %	72161.0725	9.53 %	0.5654	10.0731
RF	112592.3430	14.98 %	79145.4973	10.54 %	0.5749	11.1663
Extra Tree	111905.9375	14.89 %	78552.7607	10.45 %	0.5801	11.1023
XGBoost	113238.0113	15.07 %	76712.4545	10.19 %	0.5700	10.8046
LightGBM	116509.9906	15.50 %	85359.0804	11.38 %	0.5448	12.1455
Stacking	111382.1924	14.82 %	77898.2110	10.35 %	0.5840	11.0308
Voting	111409.8980	14.83 %	78586.6833	10.45 %	0.5838	11.1274
London Dataset						
SVR	224743.4778	39.69 %	177878.2526	34.21 %	−0.0442	35.0958
ANN	191054.2584	33.74 %	152063.8484	27.13 %	0.2454	29.2797
KNN	191562.3837	33.83 %	150445.1244	27.32 %	0.2413	29.3078
RF	189917.3260	33.54 %	150084.5000	27.16 %	0.2543	29.1307
Extra Tree	189930.2590	33.54 %	150012.3166	27.15 %	0.2542	29.0442
XGBoost	190450.5461	33.63 %	149894.4759	27.17 %	0.2501	29.1188
LightGBM	189904.2636	33.53 %	149810.4227	27.14 %	0.2544	29.0743
Stacking	189721.0286	33.50 %	149717.5285	27.08 %	0.2559	29.1205
Voting	189734.8561	33.50 %	149761.1107	27.12 %	0.2558	29.0754
Singapore Dataset						
SVR	1059199.404	57.44 %	650130.0382	41.63 %	0.0192	35.6011
ANN	330986.3821	17.95 %	169332.9713	9.11 %	0.9042	9.5089
KNN	285977.1482	15.51 %	118975.5423	6.13 %	0.9285	6.1038
RF	231217.598	12.54 %	104175.8097	5.31 %	0.9533	5.3792
Extra Tree	228230.9138	12.38 %	100394.3452	5.09 %	0.9545	5.1836
XGBoost	271368.5953	14.72 %	110316.9031	5.39 %	0.9356	5.5134
LightGBM	247973.183	13.45 %	113729.1783	5.81 %	0.9462	5.9445
Stacking	214473.2494	11.63 %	100434.4198	5.16 %	0.9598	5.2183
Voting	219820.6582	11.92 %	101856.9489	5.12 %	0.9578	5.2449

reorganization are not applicable for this column, and target encoding is used; (2) ADDRESS. The ADDRESS column and “New York” are filled into Estate Address and City/Country Name on the web app, respectively. A total of 11809 records are kept for geocoding, and 11667 valid geocoded records are downloaded for feature engineering.

London: Physical and transaction attributes include *Property Type*, *Old/New*, *Duration*, *PPD Category Type*. Property address columns include *Postcode*, *PAON*, and *Street*. The price column is *Price*. No column is needed for ordinal encoding. The first letters in the Postcode represent specific areas (e.g., “NW” in “NW7 3RX”): E-Eastern, EC-Eastern Central, N-Northern, NW-Northwestern, SE-Southwestern, W-Western, WC-Western Central. The Easycoding Chatbot is initiated to generate Python codes for extracting area information with the prompt: “The function needs to extract the strings from the first position to the first number with

single-line code.” Target encoding is used for the Postcode column since the number of unique values for this column is larger than five. A total of 50366 records are kept for geocoding, and 39681 valid geocoded records are downloaded for feature engineering.

Singapore: Physical and transaction attributes include *Area(SQFT)*, *Number of Units*, *Property Type*, *Tenure*, *Type of Sale*, *Type of Area*, *Floor Level*, and *Market Segment*. Property address columns include *Project Name*, *Street Name*, and *Postal District*. The price column is *Transacted Price (\$)*. The values in *Floor Level* are strings (e.g., “B1 to B5”, “16–20”, etc.). Therefore, ordinal column transformation is implemented by manually ranking the values in an ascending order (i.e., “B1 to B5”, “01–05”, ..., “61–65”). The *Tenure* column has many representations for leasehold, such as “99 yrs lease commencing from 2018” and “99 yrs lease commencing from 2006”. The easy-coding Chatbot is initiated to

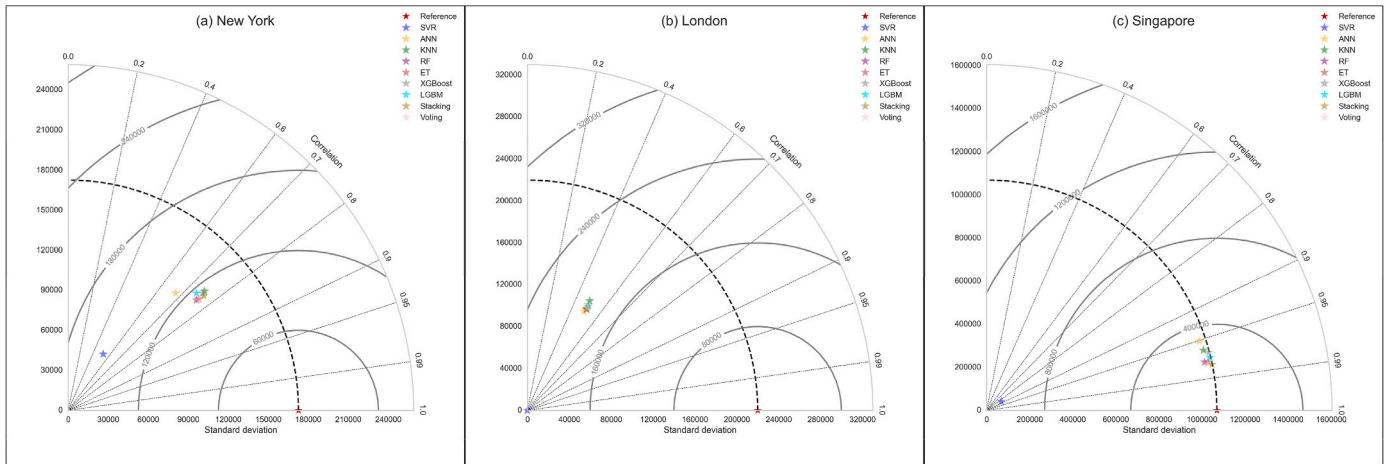


Fig. 8. Taylor diagrams of comparing the performances of base models in AutoML4RPV: (a) New York, (b) London, and (c) Singapore.

generate Python codes to change the values into “Freehold” or “Leasehold”. The prompt is set as “The function aims to transform the values in this column into freehold if the values contain freehold. Otherwise, the values are transformed into leasehold”. A total of 19,619 records are kept for geocoding, and 16125 valid geocoded records are downloaded for feature engineering.

To ensure the high accuracy and quality of OSM data, the complete OSM data for the study areas in 2022 was collected to ensure consistency in housing transaction data and road network. Then, the OSM data was compared with Google Maps in ArcGIS Pro software for manual checks regarding positional accuracy, completeness, and semantic accuracy. The comparison results show that the collected OSM road network is fairly accurate. For all datasets, six POI types are selected: bus stops, railway stations, supermarkets, malls, hospitals, and parks. The distance range is set as 1000 m. Types for the area and distance are set as circular area and Euclidean distance, respectively. Two types of line attributes are selected: primary road and railway. The polygon type is identified as *water*. The types of area and distance to calculate building density and accessibility are set as circular area and Euclidean distance. The distance range to derive polygon attributes is also set as 1000 m. The training data size and the MRMR feature selection configuration are set as default values.

4.2. Experimental validation with current AutoML frameworks

Six domain-agnostic AutoML frameworks will be compared with our proposed AutoML4RPV for comparison with AutoML4RPV regarding data preparation and model testing performances., including Tree-based Pipeline Optimization Tool (TPOT), H2O, Fast Library for Automated Machine Learning (FLAML), major-supervised, PyCaret, AutoGluon.

4.2.1. TPOT

TPOT is a Python-based AutoML tool that uses genetic programming to optimize machine learning pipelines (Olson et al., 2016). Designed for classification and regression tasks, it automates feature preprocessing, model selection, and hyperparameter tuning by exploring thousands of possible *scikit-learn* pipeline combinations, evolving them over generations to find the best-performing solution for a given dataset.

4.2.2. H2O

H2O’s AutoML automates the ML workflow by supporting the automatic training and tuning of ML models within a user-specified time limit (Ledell, 2020). It requires two types of parameters to initiate the automated ML workflow: (1) data parameters, including training data and target column name or index. (2) stopping parameters, including *max_runtime_sec* specifying the maximum time the AutoML process will

run for and *max_models* specifying the maximum number of models to build in an AutoML run except the stacked ensemble models.

4.2.3. FLAML

FLAML is a lightweight Python library for efficient automation of machine learning and AI operations using large language models and machine learning models (Wang et al., 2019). FLAML supports task-oriented AutoML using a *scikit-learn* style estimator with the standard fit and predict functions. Users should specify the training data and the task type (e.g., regression or classification).

4.2.4. Major-supervised

The mljar-supervised is a tabular data-oriented AutoML Python package designed to find the best model (Płońska and Płoński, 2021). It generates markdown reports from AutoML training, including full ML details and charts, to show users how the ML pipeline is constructed. Like FLAML, it uses a *scikit-learn* style estimator with the standard *fit* and *predict* functions. Three built-in modes are available in the mljar-supervised: (1) *Explain* when the user wants to explain and understand the data. (2) *Perform* when the user wants to train a model that will be used in real-life use cases. (3) *Compete* – when the users want to achieve maximum performance for machine learning competitions.

4.2.5. PyCaret

PyCaret is an open-source, low-code ML Python library that serves as an end-to-end ML and model management tool, automating ML workflows (Ali, 2020). PyCaret supports data preprocessing and transformation functions, including *data preparation*, *scale and transform*, *feature engineering*, and *feature selection*. Data and the target variable are required in the experiment setup phase. Different feature types, such as ordinal, numeric categorical, datetime, and text, are supported and should be specified.

4.2.6. AutoGluon

AutoGluon automates machine learning (ML) tasks and enables users to train and deploy high-accuracy models for images, text, time series, and tabular data (Erickson et al., 2020). AutoGluon supports automated feature engineering, converting raw tabular data into a format ready for ML model development. Five feature types can be detected and encoded: boolean, numerical, categorical, datetime, and text. However, it only supports filling in missing values and cannot handle outliers, which can largely affect model performances. The *AutoMLPipelineFeatureGenerator* function is used for feature engineering, after which the model training is initiated using the *TabularPredictor.fit* function.

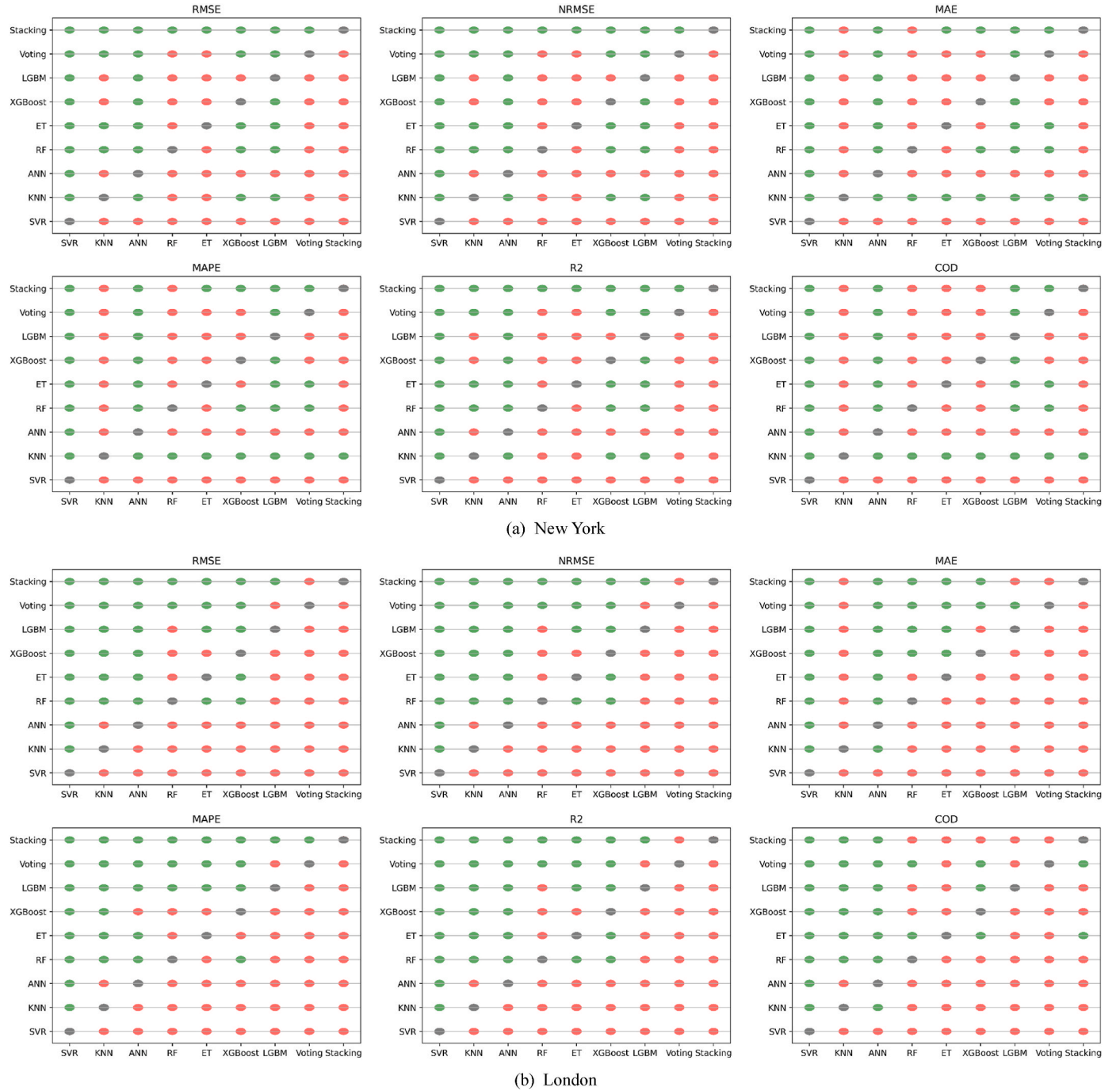


Fig. 9. Wilcoxon signed-rank test results of base modes in AutoML4RPV and benchmark ML models for datasets in: (a) New York, (b) London and (c) Singapore.

5. Experimental results analysis and discussion

5.1. Performance of the proposed AutoML4RPV

Fig. 7 shows the effects of the MRMR feature selection method on model performances measured by the R squared on the test set. The model performances for all datasets increase rapidly when fewer than ten features are selected, indicating that MRMR can effectively capture the explainable features. The R squared value of the New York dataset slightly increases with more chosen features, while that of the London and Singapore datasets remains stable afterward. AutoML4RPV achieves the highest and lowest R-squared values of approximately 0.95 and 0.25

for the Singapore and London datasets, respectively. The best performance for the New York dataset is about 0.58. The predictive models in New York and London perform worse than those in Singapore because of the floor area variable. The total housing price is calculated by multiplying the floor area and unit housing price; larger floor areas often lead to higher total prices. The correlation coefficient between floor area and housing prices in Singapore is 0.7506. However, there is a weak correlation between GSF and sale prices in New York with a correlation coefficient of 0.3023, which is probably caused by the mix of different building types in the dataset (e.g., a small-sized but high-priced condominium located in the city center and a large-sized but low-priced townhouse situated in the suburbs). London datasets have no floor

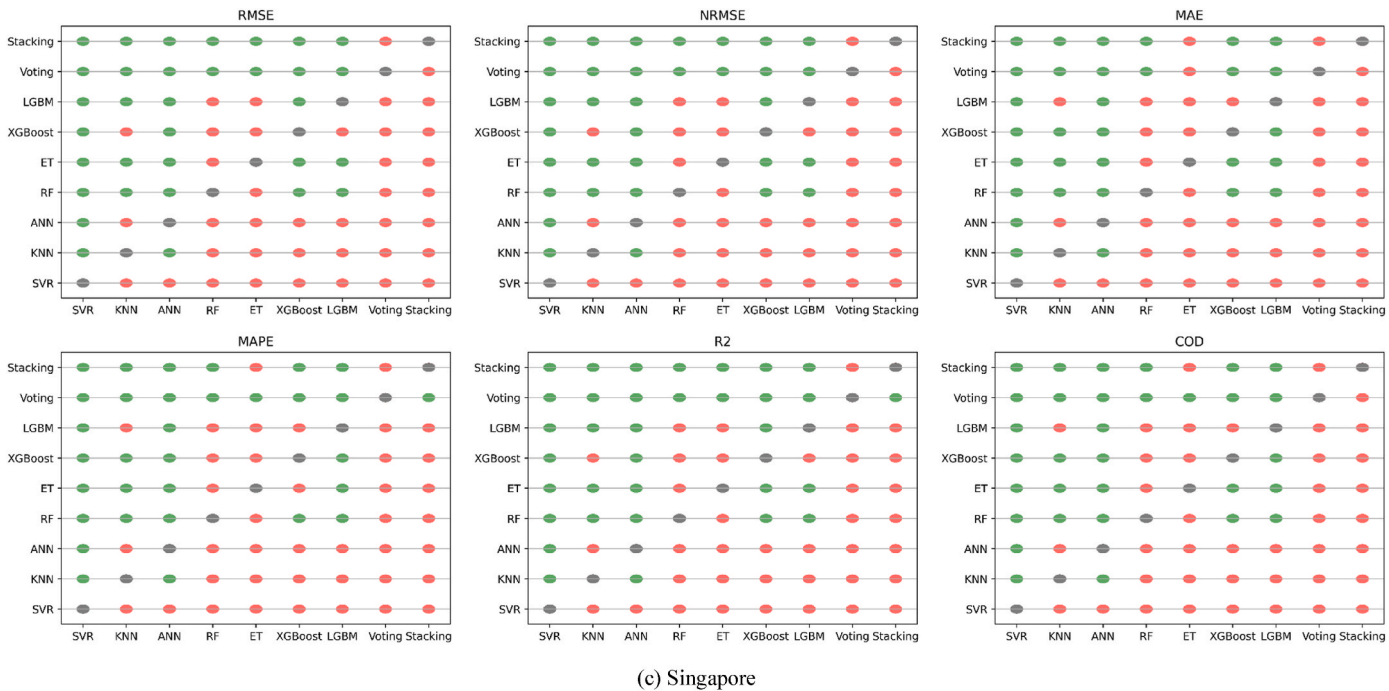


Fig. 9. (continued).

Table 5
Details of the best ML pipelines.

Description	New York	London	Singapore
Average ranking	1.5	2.5	2.0
Pipeline ID	47	18	41
No. of features	28	9	25
Selected model	Stacking	Voting	Stacking
RMSE	111382.1924	189734.8561	214473.2494
NRMSE	14.82 %	33.50 %	11.63 %
MAE	77898.2110	149761.1107	100434.4198
MAPE	10.35 %	27.12 %	5.16 %
R2	0.5840	0.2558	0.9598
COD	11.0308	29.0754	5.2183

Table 6
Data preparation and model testing performances based on raw data.

Dataset	AutoML Frameworks	No. of Finalized Records	No. of Finalized Features	Best Model Performances (R squared)
New York	PyCaret	93,427	28	0.0047
	AutoGluon	93,427	1573	0.4964
	AutoML4PV	11,667	28	0.5840
London	PyCaret	63,092	19	0.0547
	AutoGluon	63,092	15	0.2164
	AutoML4PV	39,681	9	0.2558
Singapore	PyCaret	19,652	43	0.6560
	AutoGluon	19,652	19	0.8705
	AutoML4PV	16,125	25	0.9598

area feature, which may explain the poor prediction performance. The six base models of AutoML4RPV perform differently across various study areas, highlighting the need to experiment with different ML models for specific problems, as per the no-free-lunch theorem.

Table 4 compares the best performances of different models in AutoML4RPV and three additional benchmark models: k-nearest neighbors (KNN), support vector regression (SVR), and artificial neural network Have we correctly interpreted the following funding source(s)

and country names you cited in your article: Google, United States; API, United States; DP, United States; SVR, The Netherlands; FE, United States?(ANN). The best model performance for each model type is identified in this manner: ML pipelines with the same model types are selected and ranked based on the average ranking of all evaluation metrics. The ML pipeline with the highest average ranking is the model with the best performance. The optimal values of each evaluation metric are shown in bold black. AutoML4RPV achieves the best performance in all metrics for the London and Singapore datasets, and the best RMSE, NRMSE, and R squared for the New York dataset, where KNN performs the best in terms of MAE, MAPE, and COD. Fig. 8 shows the Taylor diagrams comparing benchmark ML models and base models in AutoML4RPV for all datasets. The Taylor diagram presents standard deviation (SD), Pearson correlation coefficient (r), and root mean square error (RMSE) in a 2-D graph (Taylor, 2001). A model situated closer to the reference point has better prediction performance. The Taylor diagrams show that SVR is the worst-performing model for all datasets. Stacking models perform better than others for the New York and Singapore datasets, while there is no obvious difference among models except SVR for the London dataset.

The Wilcoxon signed-rank test is implemented to distinguish the performance between base models in AutoML4RPV and benchmark ML models through pairwise comparison. The significance plots are illustrated in Fig. 9. For the New York dataset (Fig. 9a), KNN performs significantly better than other models in terms of MAE, MAPE, and COD. The stacking model performs significantly better than other models in terms of RMSE, NRMSE, and R-squared. For the London dataset (Fig. 9b), the stacking model outperforms the others significantly in terms of MAPE. For the Singapore dataset (Fig. 9c), the voting model outperforms the others in terms of MAPE and R-squared.

The total number of ML pipelines is calculated by multiplying the number of base models in AutoML4RPV by the number of feature selection configurations. Friedman tests are implemented on these ML pipelines, and results show that the ML pipelines significantly differ in model performance for all datasets. Table 5 presents the best ML pipelines in AutoML4RPV for all datasets: (a) the best ML pipeline for the New York dataset selects 28 features and is formulated based on the stacking model; (2) the best ML pipeline for the London dataset requires

Table 7

Performance comparison using same datasets delivered by AutoML4RPV.

Evaluation metrics	RMSE	NRMSE	MAE	MAPE	R2	COD
New York Dataset						
TPOT	90675.8009	12.75 %	66354.1416	9.40 %	0.6717	9.8174
H2O	97368.1936	13.69 %	73002.5455	10.33 %	0.6215	10.7764
FLAML	110757.8168	15.57 %	84067.1508	11.84 %	0.5102	12.3984
mjar-supervised	103411.4655	14.54 %	77293.0316	10.87 %	0.5730	11.4101
PyCaret	120564.9404	16.95 %	93762.9307	13.01 %	0.4196	13.4300
AutoGluon	99763.4219	14.02 %	74699.1406	10.56 %	0.6026	11.0248
AutoML4RPV	111382.1924	14.82 %	77898.2110	10.35 %	0.5840	11.0308
London Dataset						
TPOT	189499.2179	33.15 %	151991.4182	26.95 %	0.2593	28.8265
H2O	187488.2711	32.80 %	149674.9642	26.59 %	0.2750	28.5262
FLAML	187887.7032	32.87 %	149837.5128	26.66 %	0.2719	28.4139
mjar-supervised	188031.3445	32.89 %	150042.8795	26.67 %	0.2708	28.5458
PyCaret	216309.2073	37.84 %	175337.5529	31.25 %	0.0349	34.3078
AutoGluon	187648.4256	32.83 %	149819.1719	26.61 %	0.2737	28.5612
AutoML4RPV	189734.8561	33.50 %	149761.1107	27.12 %	0.2558	29.0754
Singapore Dataset						
TPOT	230353.9094	12.80 %	104002.7313	5.42 %	0.9451	5.5078
H2O	217765.1211	12.10 %	107056.5448	5.75 %	0.9509	5.8045
FLAML	289371.9292	16.08 %	110791.5999	5.78 %	0.9133	5.7730
mjar-supervised	233116.4686	12.95 %	125253.7606	6.71 %	0.9438	6.8184
PyCaret	242253.7674	13.46 %	129557.4428	7.09 %	0.9393	7.3504
AutoGluon	195624.5926	10.87 %	104677.6953	5.73 %	0.9604	5.6882
AutoML4RPV	214473.2494	11.63 %	100434.4198	5.16 %	0.9598	5.2183
UCI Real Estate Valuation Dataset						
TPOT	6.7693	18.21 %	4.5160	11.97 %	0.7311	14.0779
H2O	7.1532	19.25 %	4.5993	12.19 %	0.6998	14.5571
FLAML	7.2537	19.52 %	5.0590	13.30 %	0.6913	16.0538
mjar-supervised	8.6109	23.17 %	5.3489	13.47 %	0.5649	16.7236
PyCaret	9.9700	26.83 %	6.9917	18.43 %	0.4167	21.2111
AutoGluon	7.0588	18.99 %	4.6038	12.23 %	0.7076	15.2080
AutoML4RPV	5.2207	13.89 %	3.9823	10.66 %	0.8246	10.8215

nine features and is built on top of the voting model. (3) the best ML pipeline for the Singapore dataset utilizes 25 features and the stacking model to achieve the best performance. The results show that stacking and voting could perform better than the individual ML model, indicating the necessity and advantages of ensemble learning implementation.

5.2. Comparison with existing AutoML frameworks

5.2.1. Data preparation performance with raw data

As AutoGluon and PyCaret free users from providing clean data after data preprocessing for model training, these two AutoML frameworks are compared with AutoML4RPV by feeding them raw data and initiating the model training. Table 6 presents the results of data preparation and model testing performance. PyCaret and AutoGluon keep all the records for all datasets. AutoML4RPV keeps fewer records because it directly removes missing and inconsistent values and outliers. AutoGluon creates more than 1000 features for the New York dataset because it deals with text columns in the dataset (i.e., ADDRESS column in the New York dataset) in these two ways (Erickson et al., 2020): (1) using an n-gram feature generator to extract n-grams (short strings) from the text feature and add many additional columns for n-gram feature. (2) calculating and adding additional numerical features, such as word counts, character counts, proportion of uppercase characters, etc. AutoML4RPV creates the fewest features for the London dataset and the same number of features as PyCaret for the New York dataset. AutoML4RPV achieves the highest R-squared value because it is domain-specific and can produce interpretable features, such as latitude and longitude, derived from the property address. However, transforming the proper address into text-related features does not significantly improve predictive performance.

5.2.2. Model training performance

To evaluate the performance of AutoML4RPV's domain-agnostic modules, the same training and test data used by AutoML4PV (New York, London, and Singapore) are employed for a performance comparison between AutoML4PV and six AutoML frameworks (TPOT, H2O, FLAML, mjar-supervised, PyCaret, and AutoGluon) with default parameters. A real estate valuation dataset (Yeh and Hsu, 2018) from the UC Irvine (UCI) machine learning repository is also included for performance comparison, and it contains 414 instances and six features. The model training performance results of the seven AutoML frameworks using four datasets are shown in Table 7. The optimal values of all evaluation metrics are presented in bold black. TPOT performs best in the New York dataset for all evaluation metrics. H2O performs best in the London dataset for all evaluation metrics except COD, and FLAML has the optimal COD value. For the Singapore dataset, AutoML4RPV performs best in terms of MAE, MAPE, and COD, while AutoGluon performs best regarding RMSE, NRMSE, and R-squared. AutoML4RPV performs best in all evaluation metrics for the public valuation dataset. Fig. 10 shows the Taylor diagrams for all datasets, comparing AutoML4RPV with six AutoML frameworks. The predictions by AutoML4RPV have similar SD values to the reference point for the New York, London, and Singapore datasets. AutoML4RPV is the closest model to the reference point for the UCI valuation dataset, indicating that it is the best-performing model for predicting prices in this dataset, which is consistent with the previous discussion.

Table 8 shows the average rankings of AutoML4RPV and six benchmark AutoML frameworks for the four datasets. H2O has the highest average ranking of model performances, followed by TPOT and AutoGluon. The Friedman test is implemented, and the resultant *p*-value is 0.024, indicating the performances of the seven AutoML frameworks are different at the 95 % confidence level. The Nemenyi test is applied to

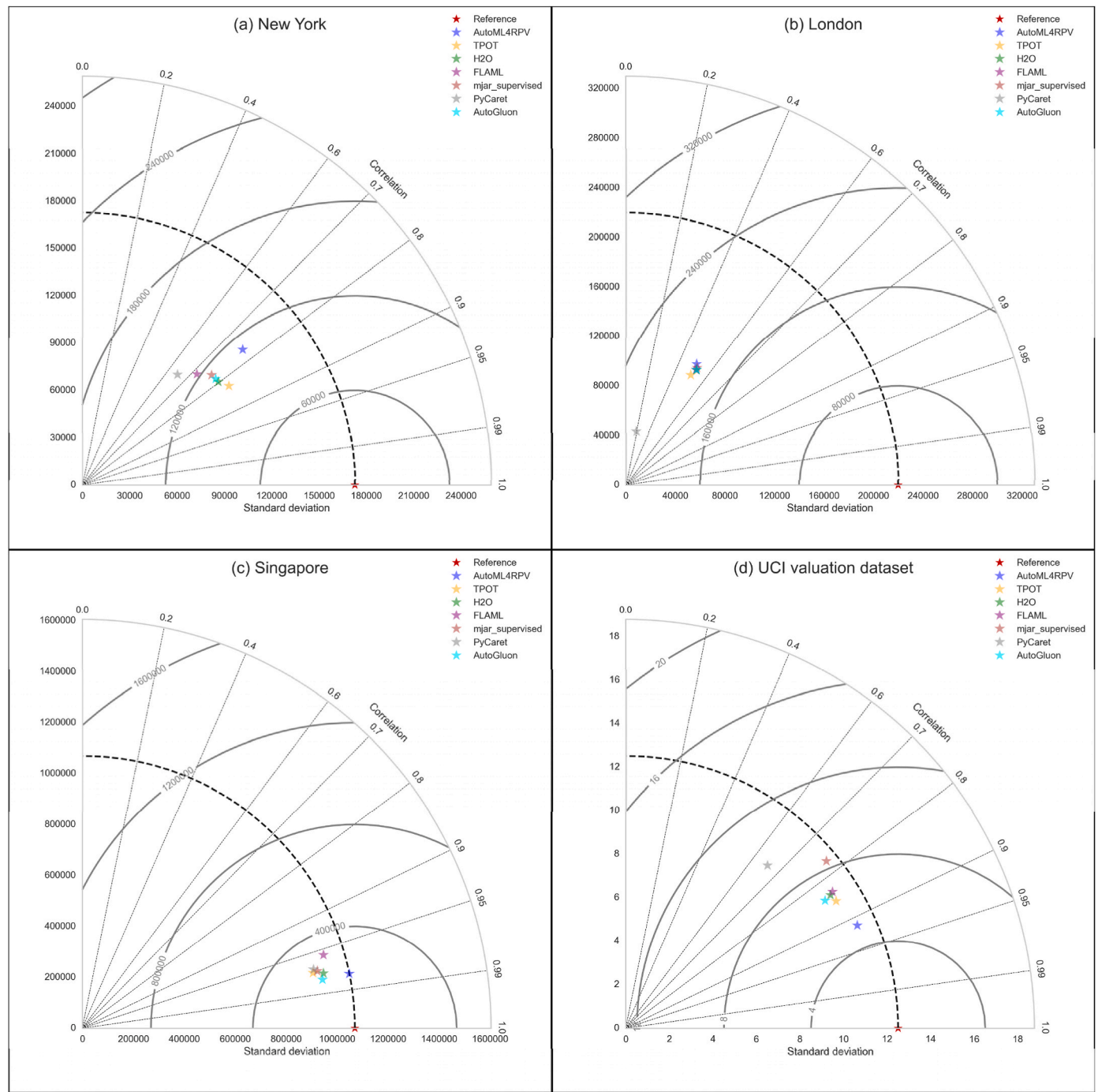


Fig. 10. Taylor diagrams of comparing AutoML4RPV and benchmark AutoML frameworks for: (a) New York, (b) London, (c) Singapore, and (d) UCI valuation dataset.

Table 8					
Average rankings of AutoML4RPV and benchmark AutoML frameworks.					
	New York	London	Singapore	UCI valuation dataset	Average ranking
AutoML4RPV	4.5	5.3	1.5	1	3.1
TPOT	1	5.2	3	2	2.8
H2O	2	1.2	3.7	3.5	2.6
FLAML	5.8	2.8	5.8	5	4.9
mjar_supervised	4.5	4	5.5	6	5.0
PyCaret	7	7	6.5	7	6.9
AutoGluon	3.2	2.5	2	3.5	2.8

further distinguish between the AutoML frameworks. In this study, the number of datasets (N) is four, and the number of models (k) is seven. The critical value $q_{0.1}$ is 2.693, and the corresponding CD value is calculated at 4.1. Fig. 11 shows the critical difference diagram of the Nemenyi posthoc test at a significance level of 0.1: black points refer to the average ranking of different AutoML frameworks, and the horizontal line segment crossing black points represents the range of CD values. The results show that the performances of H2O, TPOT, and AutoGluon are significantly better than those of PyCaret because their rank differences are larger than the CD value. Our proposed AutoML4RPV has an average ranking of 3.1, and there is no statistically significant difference in performance among H2O, TPOT, AutoGluon, AutoML4RPV, FLAML,

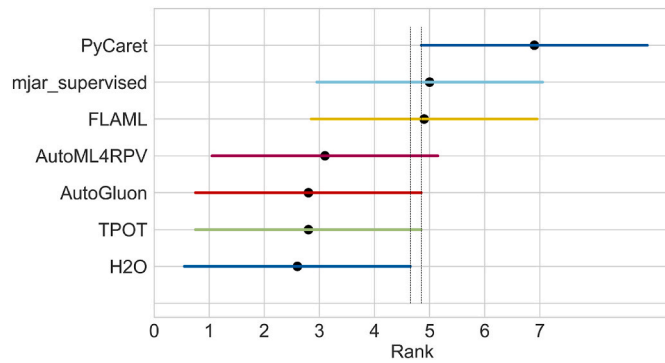


Fig. 11. The Nemenyi test results among AutoML frameworks.

and mjar_supervised.

5.3. Implications of this study

5.3.1. Theoretic implications

The proposed AutoML4RPV is an efficient automation tool for developing an ML-based housing price prediction model. It has several theoretical implications for residential property valuation: **(1) Roles of domain knowledge.** Incorporating domain knowledge into data collection and preparation stages can produce high-quality data and extract meaningful features to improve prediction accuracy. **(2) Hybrid**

ML modeling strategies. A hybrid of ML models with ensemble learning strategies (i.e., stacking and voting) produces better performance than individual ones. **(3) Generalization capability to other domains.** The domain-agnostic modules of AutoML4RPV include the proposed modeling strategies and other domain-agnostic AutoML framework alternatives. The modules can be utilized separately to provide more reliable and accurate predictions for regression tasks across various domains, thereby enhancing the generalization capability of AutoML4RPV.

5.3.2. Practical implications

This study developed a web application based on the Python *Shiny* package to enable users with no programming experience to build, compare, and automate end-to-end ML models more easily. Fig. 12 presents the system architecture of AutoML4RPV's ultimate version. The architecture comprises the hardware, software, database, service, and presentation layers. **The hardware and software layers** provide physical and software components that provide the foundation for the development, deployment, and management of AI projects, including (1) hardware components such as cloud servers, cloud databases, and cloud computing and (2) python libraries regarding spatial data processing (*GeoPandas*), network analysis (*NetworkX*), machine learning (*Scikit Learn*), and deep learning (*Pytorch*). **The database layer** includes business data and system data. Business data include structured, non-structured, and geospatial data. System data include data processing, GIS analytics algorithms, machine learning, and deep learning models. **The service layer** is the core layer, and it provides two main types of

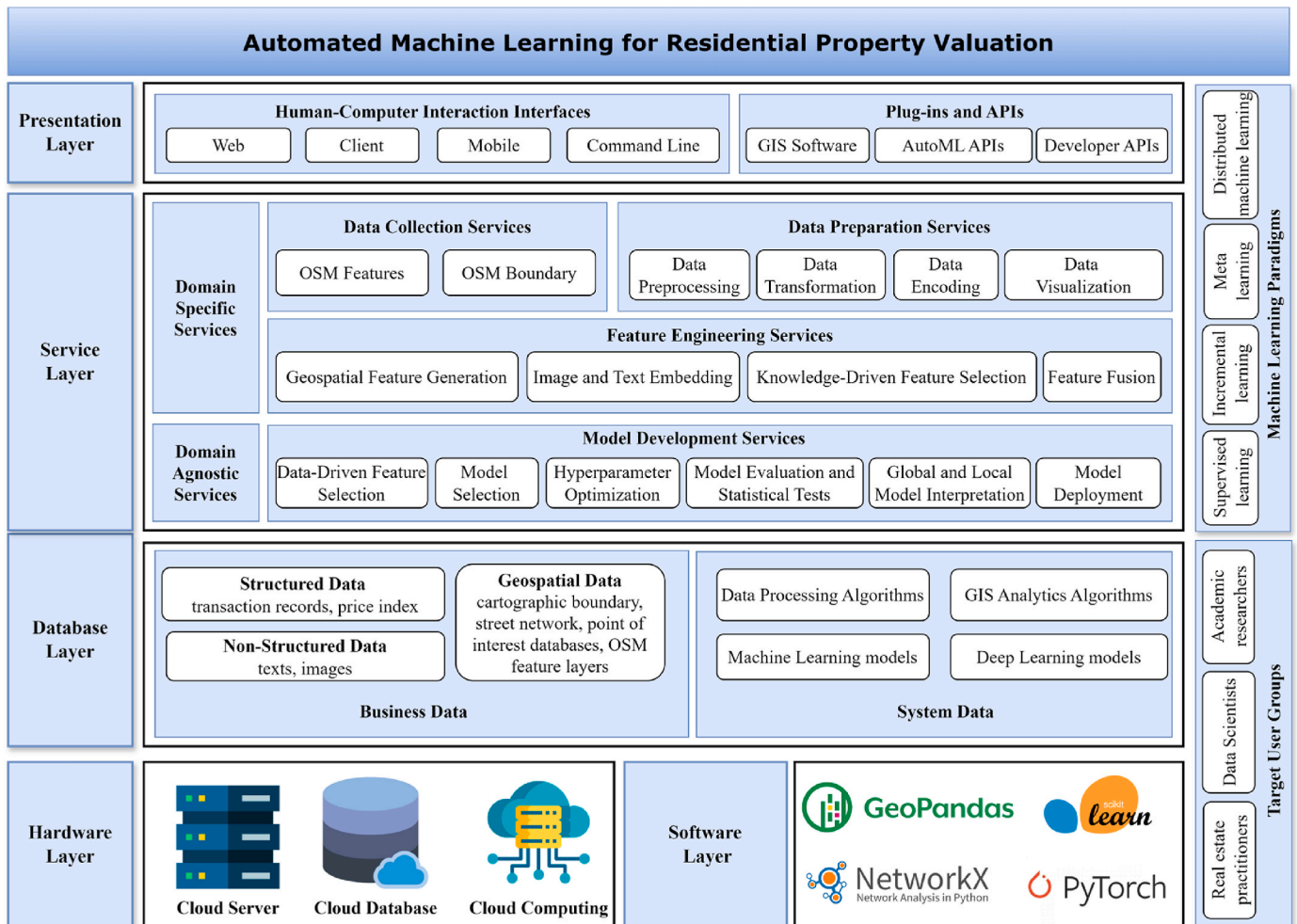


Fig. 12. System architecture of the AutoML4RPV framework.

services: (1) domain-specific services, including data collection and preparation and feature engineering, and (2) domain-agnostic services related to model development. The **presentation layer** encompasses various human-computer interaction interfaces, including web, client, mobile, and command-line interfaces. It can also serve as plug-ins for GIS software, as well as developer APIs for available AutoML platforms and AutoML4RPV. Users of AutoML4PV involve real estate practitioners, data scientists, and academic researchers. AutoML4PV will support varied machine learning paradigms such as supervised learning for regression and classification tasks, incremental learning for better adaption to dynamic market conditions using streaming transaction data, meta-learning for learning to learn, and distributed machine learning that trains a series of ML pipelines on different workers or trains a shared model without exchanging raw data (federated learning).

6. Limitations and future research directions

6.1. Limitations of this study

Despite the effectiveness of our proposed AutoML4RPV framework, it has some limitations: (1) **Inconsistent data quality and availability of volunteered geographic information (VGI)**. OSM is a free, open map database updated and maintained by a community of volunteers. Therefore, the level of detail in OSM data may be inconsistent across regions due to varying levels of volunteer contributions or data availability, which could bias the results, particularly in cities with less complete OSM coverage. Alternative geographic information system (GIS) platforms such as Google Maps could be utilized. (2) **Incomplete feature mining**. The predictive models in London perform worse than other study areas. It is necessary to extract more explanatory variables from other public data sources (e.g., real estate agency websites) to increase model performance. (3) **Time-consuming data preparation and feature engineering process**. Compared to the model generation and evaluation process, geocoding (in the data preparation stage) and feature generation (in the feature engineering stage) take longer time to proceed than the model generation and evaluation process. More advanced techniques should be employed to enhance the scalability of AutoML4RPV, including cloud computing and distributed computing.

6.2. Future research directions

Further improvements of AutoML4RPV can be conducted in terms of data, features, models, human-computer interaction, and generalization capability verification: (1) varied data imputation (Sun et al., 2023) and outlier detection methods (Boukerche et al., 2020) can be incorporated into AutoML4RPV to increase the model performance further. (2) varied feature selection methods (Solorio-Fernández et al., 2020) and dimensionality reduction methods (Ray et al., 2021) can be supported by AutoML4RPV to identify explainable feature subsets for model training (3) varied types of base learners (e.g., shallow ML models and DL models). can be incorporated in AutoML4RPV to formulate the stacking and voting framework. Ensemble pruning methods (Martínez-Muñoz et al., 2009) can be employed to reduce the ensemble of predictive models, thereby enhancing both computational efficiency and predictive performance. Hybrid DL models (Bilal et al., 2024a; Bilal et al., 2025) can also be incorporated to enrich the algorithm library of AutoML4RPV. (4) The large language model (LLM) can be further utilized to develop LLM agents to initiate the ML pipelines automatically based on the AutoML4RPV framework. (5) Housing transaction datasets in multiple regions can be collected to invite academic researchers and industry stakeholders to verify the generalization capability of domain-specific modules of AutoML4RPV. Publicly available datasets in other domains, such as concrete strength prediction (Zhang et al., 2021, 2022, 2023), building energy analysis (Mohandes et al., 2019; Zubair and Zhang, 2020), and heart disease diagnosis (Bilal et al., 2024b) can be used for algorithm benchmark to verify the generalization capability

of domain-agnostic modules of AutoML4RPV.

7. Conclusions

Building ML models for residential property valuation can be time-consuming and complex for real estate practitioners without much data science experience. It involves several key steps, including data collection and processing, feature engineering, model generation and evaluation, and model deployment. AutoML automates these steps to make machine learning more accessible to practitioners who lack extensive data science expertise. Most AutoML frameworks aim to provide a general training platform for data in any field without considering the domain knowledge in the data processing and feature engineering stages. Otherwise, the finalized data and features are meaningless as they are generated purely based on data rules, regardless of the meanings of the data and features. A comprehensive AutoML framework should be domain-specific in terms of data processing and feature engineering, and domain-agnostic in terms of automatic model training after the data and features are finalized. This study proposes AutoML4RPV by incorporating domain-specific and domain-agnostic function modules. A web app was developed to enable users without programming experience to easily build, compare, and deploy ML models from scratch. Three housing datasets in three financial centers (New York, London, and Singapore) and one public valuation dataset are used for experimental validation. Results show that our proposed AutoML4RPV framework is highly competitive regarding domain-specific and domain-agnostic services. The system architecture of the AutoML4PV's ultimate version is proposed, paving the way for further improvements of AutoML4RPV.

CRediT authorship contribution statement

Lin Deng: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xueqing Zhang:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.engappai.2025.112035>.

Data availability

The data and codes of AutoML4RPV are openly available at <https://github.com/Linhkust/AutoML4RPV>.

References

- Alfaro-Navarro, J.-L., Cano, E.L., Alfaro-Cortés, E., García, N., Gámez, M., Larraz, B., 2020. A fully automated adjustment of ensemble methods in machine learning for modeling complex real estate systems. *Complexity* 2020, 1–12. <https://doi.org/10.1155/2020/5287263>.
- Ali, M., 2020. PyCaret: an open source, low-code machine learning library in python. Retrieved from. <https://www.pycaret.org>.
- Alsharif, A., Aggarwal, K., Sonia, Kumar, M., Mishra, A., 2022. Review of ML and AutoML solutions to forecast time-series data. *Arch. Comput. Methods Eng.* 29 (7), 5297–5311. <https://doi.org/10.1007/s11831-022-09765-0>.
- Baymurzina, D., Golikov, E., Burtsev, M., 2022. A review of neural architecture search. *Neurocomputing* 474, 82–93. <https://doi.org/10.1016/j.neucom.2021.12.014>.
- Bilal, H., Muhammad, Y., Ullah, I., Garg, S., Choi, B.J., Hassan, M.M., 2025. Identification and diagnosis of chronic heart disease: a deep learning-based hybrid

- approach. *Alex. Eng. J.* 124 (February), 470–483. <https://doi.org/10.1016/j.aej.2025.03.025>.
- Bilal, H., Obaidat, M.S., Aslam, M.S., Zhang, J., Yin, B., Mahmood, K., 2024a. Online fault diagnosis of industrial robot using IoT and hybrid deep learning techniques: an experimental approach. *IEEE Internet Things J.* 11 (19), 31422–31437. <https://doi.org/10.1109/JIOT.2024.3418352>.
- Bilal, H., Tian, Y., Ali, A., Muhammad, Y., Yahya, A., Izneid, B.A., Ullah, I., 2024b. An intelligent approach for early and accurate predication of cardiac disease using hybrid artificial intelligence techniques. *Bioengineering* 11 (12). <https://doi.org/10.3390/bioengineering11121290>.
- Bischi, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.L., Deng, D., Lindauer, M., 2023. Hyperparameter optimization: foundations, algorithms, best practices, and open challenges. *Wiley Interdisc. Rev.: Data Min. Knowl. Discov. ML*. <https://doi.org/10.1002/widm.1484>.
- Bonidia, R.P., Santos, A.P.A., De Almeida, B.L.S., Stadler, P.F., Da Rocha, U.N., Sanches, D.S., De Carvalho, A.C.P.L.F., 2022. BioAutoML: automated feature engineering and metalearning to predict noncoding RNAs in bacteria. *Briefings Bioinform.* 23 (4), 1–13. <https://doi.org/10.1093/bib/bbac218>.
- Boukerche, A., Zheng, L., Alfandi, O., 2020. Outlier detection: methods, models, and classification. *ACM Comput. Surv.* 53 (3). <https://doi.org/10.1145/3381028>.
- Breiman, L., 1996. Bagging predictors. *Mach. Learn.* 24 (2), 123–140. <https://doi.org/10.1007/BF00058655>.
- Chandrashekar, G., Sahin, F., 2014. A survey on feature selection methods. *Comput. Electr. Eng.* 40 (1), 16–28. <https://doi.org/10.1016/j.compeleceng.2013.11.024>.
- Chen, T., Guestrin, C., 2016. XGBoost: a scalable tree boosting system. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 13–17 Aug., pp. 785–794. <https://doi.org/10.1145/2939672.2939785>.
- Chen, W., Zhang, L., 2022. An automated machine learning approach for earthquake casualty rate and economic loss prediction. *Reliab. Eng. Syst. Saf.* 225 (June), 108645. <https://doi.org/10.1016/j.res.2022.108645>.
- Choy, L.H.T., Ho, W.K.O., 2023. The use of machine learning in real estate research. *Land* 12 (4). <https://doi.org/10.3390/land12040740>.
- Deng, L., 2025. Real estate valuation with multi-source image fusion and enhanced machine learning pipeline. *PLoS One* 20 (5 May), 1–32. <https://doi.org/10.1371/journal.pone.0321951>.
- Deng, L., Zhang, X., 2025. Boosting the accuracy of property valuation with ensemble learning and explainable artificial intelligence: the case of Hong Kong. *Ann. Reg. Sci.* 74 (1), 1–28. <https://doi.org/10.1007/s00168-025-01365-7>.
- Deppner, J., von Ahlefeldt-Dehn, B., Beracha, E., Schaefer, W., 2023. Boosting the accuracy of commercial real estate appraisals: an interpretable machine learning approach. *J. R. Estate Finance Econ.* <https://doi.org/10.1007/s11146-023-09944-1> (Issue 0123456789). Springer US.
- Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., Smola, A., 2020. AutoGluon-Tabular: robust and accurate AutoML for structured data. <http://arxiv.org/abs/2003.06505>.
- Feurer, M., Eggenberger, K., Falkner, S., Lindauer, M., Hutter, F., 2022. Auto-sklearn 2.0: hands-free AutoML via meta-learning. *J. Mach. Learn. Res.* 23, 1–61.
- Feurer, M., Tobias Springenberg, J., Klein, A., Blum, M., Eggenberger, K., Hutter, F., 2015. Efficient and robust automated machine learning. <https://papers.nips.cc/paper/2015/hash/11d0e6287202fced83f79975ec59a3a6-Abstract.html>.
- Francia, M., Giovanelli, J., Pisano, G., 2023. HAMLET: a framework for Human-centered AutoML via structured argumentation. *Future Gener. Comput. Syst.* 142, 182–194. <https://doi.org/10.1016/j.future.2022.12.035>.
- Frazier, P.I., 2018. A tutorial on bayesian optimization. Section 5, 1–22. <http://arxiv.org/abs/1807.02811>.
- Friedman, J.H., 2001. Greedy function approximation: a gradient boosting machine. *Ann. Stat.* 29 (5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>.
- Gao, Q., Shi, V., Pettit, C., Han, H., 2022. Property valuation using machine learning algorithms on statistical areas in greater Sydney, Australia. *Land Use Policy* 123 (October), 106409. <https://doi.org/10.1016/j.landusepol.2022.106409>.
- Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., 2022a. AMLBID: an auto-explained automated machine learning tool for big industrial data. *SoftwareX* 17, 100919. <https://doi.org/10.1016/j.softx.2021.100919>.
- Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A., 2022b. Using meta-learning for automated algorithms selection and configuration: an experimental framework for industrial big data. *J. Big Data* 9 (1). <https://doi.org/10.1186/s40537-022-00612-4>.
- Garouani, M., Bouneffa, M., Ahmad, A., Hamlich, M., 2023. Version [2 . 0] - [AMLBID: An auto-explained Automated Machine Learning tool for Big Industrial Data]. *SoftwareX* 23, 101444. <https://doi.org/10.1016/j.softx.2023.101444>.
- Geerts, M., vanden Broucke, S., De Weert, J., 2023. A survey of methods and input data types for house price prediction. *ISPRS Int. J. GeoInf.* 12 (Issue 5). <https://doi.org/10.3390/ijgi12050200>.
- Geurts, P., Ernst, D., Wehenkel, L., 2006. Extremely randomized trees. *Mach. Learn.* 63 (1), 3–42. <https://doi.org/10.1007/s10994-006-6226-1>.
- Gudzius, P., Kurasova, O., Darulis, V., Filatovas, E., 2023. AutoML-Based neural architecture search for object recognition in satellite imagery. *Remote Sens.* 15 (1). <https://doi.org/10.3390/rs15010091>.
- He, X., Zhao, K., Chu, X., 2021. AutoML: a survey of the state-of-the-art. *Knowl. Base Syst.* 212, 106622. <https://doi.org/10.1016/j.knsys.2020.106622>.
- International Association of Assessing Officers, 2018. *Standard on automated valuation models (AVMs) international association of assessing officers* (Issue July). www.iaao.org.
- Jaafar, Y., Luc Laurent, J., Deruyver, A., Saber Naceur, M., 2019. Reinforcement learning for neural architecture search: a review. *Image Vis Comput.* 89, 57–66. <https://doi.org/10.1016/j.imavis.2019.06.005>.
- Kang, Y., Zhang, F., Peng, W., Gao, S., Rao, J., Duarte, F., Ratti, C., 2021. Understanding house price appreciation using multi-source big geo-data and machine learning. *Land Use Policy* 111 (September 2019), 104919. <https://doi.org/10.1016/j.landusepol.2020.104919>.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y., 2017. LightGBM: a highly efficient gradient boosting decision tree. *Adv. Neural Info. Process. Syst.* 2017-Decem(Nips) 3147–3155.
- Koh, E.J.Y., Amini, E., Gaur, S., Becerra Maquieira, M., Jara Heck, C., McLachlan, G.J., Beaton, N., 2022. An automated machine learning (AutoML) approach to regression models in minerals processing with case studies of developing industrial comminution and flotation models. *Miner. Eng.* 189 (October), 107886. <https://doi.org/10.1016/j.mineng.2022.107886>.
- Komer, B., Bergstra, J., Eliasmith, C., 2014. Hyperopt-Sklearn: automatic hyperparameter configuration for scikit-learn. In: *Proceedings of the 13th Python in Science Conference, Scipy*, pp. 32–37. <https://doi.org/10.25080/majora-14bd3278-006>.
- Kotthoff, L., Thornton, C., Hutter, F., 2017. User Guide for Auto-WEKA Version 2 . 6. <https://doi.org/10.1016/j.mineng.2022.107886>.
- Lim, Z., Bellotti, A., 2021. Normalized nonconformity measures for automated valuation models. *Expert Syst. Appl.* 180 (May), 115165. <https://doi.org/10.1016/j.eswa.2021.115165>.
- Lundberg, S.M., Lee, S.-I., 2017. A unified approach to interpreting model predictions. *Adv. Neural Inf. Process. Syst.* 30. In: <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43df28b67767-Abstract.html>.
- Martinez-Muñoz, G., Hernández-Lobato, D., Suarez, A., 2009. An analysis of ensemble pruning techniques based on ordered aggregation. *IEEE Trans. Pattern Anal. Mach. Intell.* 31 (2), 245–259. <https://doi.org/10.1109/TPAMI.2008.78>.
- Meisenbacher, S., Turowski, M., Phipps, K., Rätz, M., Müller, D., Hagenmeyer, V., Mikut, R., 2022. Review of automated time series forecasting pipelines. *Wiley Interdisc. Rev.: Data Min. Knowl. Discov.* 12 (6). <https://doi.org/10.1002/widm.1475>.
- Mohan, B., Badra, J., 2023. A novel automated SuperLearner using a genetic algorithm-based hyperparameter optimization. *Adv. Eng. Software* 175 (October 2022), 103358. <https://doi.org/10.1016/j.advengsoft.2022.103358>.
- Mohandes, S.R., Zhang, X., Mahdiyar, A., 2019. A comprehensive review on the application of artificial neural networks in building energy analysis. *Neurocomputing* 340, 55–75. <https://doi.org/10.1016/j.neucom.2019.02.040>.
- Mohr, F., Wever, M., Hüllermeier, E., 2018. ML-Plan: automated machine learning via hierarchical planning. *Mach. Learn.* 107 (8–10), 1495–1515. <https://doi.org/10.1007/s10994-018-5735-z>.
- Molnar, C., 2020. Interpretable machine learning. A guide for making black box models explainable. Book 247. <https://christophm.github.io/interpretable-ml-book>.
- Olson, R.S., Edu, O., Moore, J.H., 2016. TPOT: a tree-based pipeline optimization tool for automating machine learning. In: *Proceedings of the Workshop on Automatic Machine Learning*, vol. 64. PMLR, pp. 66–74.
- Płóńska, A., Płóński, P., 2021. MLJAR: state-of-the-art automated machine learning framework for tabular data. Retrieved from, Version 0.10.3. <https://github.com/mljar/mljar-supervised>.
- Ray, P., Reddy, S.S., Banerjee, T., 2021. Various dimension reduction techniques for high dimensional data analysis: a review. *Artif. Intell. Rev.* 54 (Issue 5). <https://doi.org/10.1007/s10462-020-09928-0>.
- Rosen, S., 1974. Hedonic prices and implicit markets: product differentiation in pure competition. *J. Polit. Econ.* 82 (1), 34–55. <https://doi.org/10.1086/260169>.
- Sagi, O., Rokach, L., 2018. Ensemble learning: a survey. *Wiley Interdisc. Rev.: Data Min. Knowl. Discov.* 8 (4), 1–18. <https://doi.org/10.1002/widm.1249>.
- Sahin, E.K., Demir, S., 2023. Greedy-AutoML: a novel greedy-based stacking ensemble learning framework for assessing soil liquefaction potential. *Eng. Appl. Artif. Intell.* 119 (November 2022), 105732. <https://doi.org/10.1016/j.engappai.2022.105732>.
- Schapire, R.E., 1990. The strength of weak learnability. *Mach. Learn.* 5 (2), 197–227. <https://doi.org/10.1023/A:1022648800760>.
- Shi, X., Wong, Y.D., Chai, C., Li, M.Z.F., 2021. An automated machine learning (AutoML) method of risk prediction for decision-making of autonomous vehicles. *IEEE Trans. Intell. Transport. Syst.* 22 (11), 7145–7154. <https://doi.org/10.1109/TITS.2020.3002419>.
- Smith, M.J., Sala, C., Kanter, J.M., Veeramachaneni, K., 2020. The machine learning bazaar: harnessing the ML ecosystem for effective system development. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 785–800. <https://doi.org/10.1145/3318464.3386146>.
- Solorio-Fernández, S., Carrasco-Ochoa, J.A., Martínez-Trinidad, J.F., 2020. A review of unsupervised feature selection methods. *Artif. Intell. Rev.* 53 (2), 907–948. <https://doi.org/10.1007/s10462-019-09682-y>.
- Soltani, A., Heydari, M., Aghaei, F., Pettit, C.J., 2022. Housing price prediction incorporating spatio-temporal dependency into machine learning algorithms. *Cities* 131 (September 2021), 103941. <https://doi.org/10.1016/j.cities.2022.103941>.
- Soltani, A., Pettit, C.J., Heydari, M., Aghaei, F., 2021. Housing price variations using spatio-temporal data mining techniques. *J. Hous. Built Environ.* 36 (3), 1199–1227. <https://doi.org/10.1007/s10901-020-09811-y>.
- Sun, Y., Li, J., Xu, Y., Zhang, T., Wang, X., 2023. Deep learning versus conventional methods for missing data imputation: a review and comparative study. *Expert Syst. Appl.* 227 (April), 120201. <https://doi.org/10.1016/j.eswa.2023.120201>.
- Swearingen, T., Drevo, W., Cyphers, B., Cuesta-Infante, A., Ross, A., Veeramachaneni, K., 2017. ATM: a distributed, collaborative, scalable system for automated machine learning. In: *Proceedings - 2017 IEEE International Conference on Big Data, Big Data 2017*, 2018-Janua, pp. 151–162. <https://doi.org/10.1109/BigData.2017.8257923>.
- Taylor, K.E., 2001. Summarizing multiple aspects of model performance in a single diagram. *J. Geophys. Res. Atmos.* 106, 7183–7192.

- Tekouabou, S.C.K., Gherghina, Ş.C., Kameni, E.D., Filali, Y., Gartoumi, K.I., 2023. AI-Based on machine learning methods for urban real estate prediction: a systematic survey. *Arch. Comput. Methods Eng.* <https://doi.org/10.1007/s11831-023-10010-5>, 0123456789.
- Thakur, A., Krohn-Grimberghe, A., 2015. AutoCompete: a framework for machine learning competition. <http://arxiv.org/abs/1507.02188>.
- Thomas, J., Coors, S., Bischl, B., 2018. Automatic gradient boosting, 2013. <http://arxiv.org/abs/1807.03873>.
- Thornton, C., Hutter, F., Hoos, H.H., Leyton-Brown, K., 2013. Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 847–855. <https://doi.org/10.1145/2487575.2487629>.
- Wang, C., Wu, Q., Weimer, M., Zhu, E., 2019. FLAML: a fast and lightweight AutoML library. <http://arxiv.org/abs/1911.04706>.
- Wardle, M., Mainelli, M., 2023. The Global Financial Centres Index 17, vol. 56. March. http://www.longfinance.net/images/GFCI15_15March2014.pdf.
- Waring, J., Lindvall, C., Umeton, R., 2020. Automated machine learning: review of the state-of-the-art and opportunities for healthcare. *Artif. Intell. Med.* 104 (October 2019), 101822. <https://doi.org/10.1016/j.artmed.2020.101822>.
- Wen, R., Li, S., 2023. Spatial decision support systems with automated machine learning: a review. *ISPRS Int. J. GeoInf.* 12 (1). <https://doi.org/10.3390/ijgi12010012>.
- Wever, M., Tornede, A., Mohr, F., Hullermeier, E., 2021. AutoML for multi-label classification: overview and empirical evaluation. *IEEE Trans. Pattern Anal. Mach. Intell.* 43 (9), 3037–3054. <https://doi.org/10.1109/TPAMI.2021.3051276>.
- Wolpert, D., 1992. Stacked generalization (stacking). *Neural Netw.* 5 (2), 241–259.
- Xu, L., Li, Z., 2021. A new appraisal model of second-hand housing prices in China's first-tier cities based on machine learning algorithms. *Comput. Econ.* 57 (2), 617–637. <https://doi.org/10.1007/s10614-020-09973-5>.
- Yacim, J.A., Boshoff, D.G.B., 2018. Impact of artificial neural networks training algorithms on accurate prediction of property values. *J. R. Estate Res.* 40 (3), 375–418. <https://doi.org/10.1080/10835547.2018.12091505>.
- Yao, Q., Wang, M., Chen, Y., Dai, W., Li, Y.-F., Tu, W.-W., Yang, Q., Yu, Y., 2018. Taking human out of learning applications: a survey on automated machine learning, 1–20. <http://arxiv.org/abs/1810.13306>.
- Yeh, I.C., Hsu, T.K., 2018. Building real estate valuation models with comparative approach through case-based reasoning. *Appl. Soft Comput.* 65, 260–271.
- Yoo, S., Im, J., Wagner, J.E., 2012. Variable selection for hedonic model using machine learning approaches: a case study in Onondaga county, NY. *Landsc. Urban Plann.* 107 (3), 293–306. <https://doi.org/10.1016/j.landurbplan.2012.06.009>.
- Zhan, C., Liu, Y., Wu, Z., Zhao, M., Chow, T.W.S., 2023. A hybrid machine learning framework for forecasting house price. *Expert Syst. Appl.* 233 (June). <https://doi.org/10.1016/j.eswa.2023.120981>.
- Zhang, X., Akber, M.Z., Zheng, W., 2021. Prediction of seven-day compressive strength of field concrete. *Constr. Build. Mater.* 305 (August), 124604. <https://doi.org/10.1016/j.conbuildmat.2021.124604>.
- Zhang, X., Akber, M.Z., Zheng, W., 2022. Predicting the slump of industrially produced concrete using machine learning: a multiclass classification approach. *J. Build. Eng.* 58 (July), 104997. <https://doi.org/10.1016/j.jobte.2022.104997>.
- Zhang, X., Zeshan Akber, M., Poon, C.S., Zheng, W., 2023. Predicting the 28-day compressive strength by mix proportions: insights from a large number of observations of industrially produced concrete. *Constr. Build. Mater.* 400 (February), 132754. <https://doi.org/10.1016/j.conbuildmat.2023.132754>.
- ZhangZhou, J., He, C., Sun, J., Zhao, J., Lyu, Y., Wang, S., Zhao, W., Li, A., Ji, X., Agarwal, A., 2024. Geochemistry π : automated machine learning python framework for tabular data. *G-cubed* 25 (1), 1–14. <https://doi.org/10.1029/2023GC011324>.
- Zubair, M.U., Zhang, X., 2020. Explicit data-driven prediction model of annual energy consumed by elevators in residential buildings. *J. Build. Eng.* 31 (February), 101278. <https://doi.org/10.1016/j.jobte.2020.101278>.